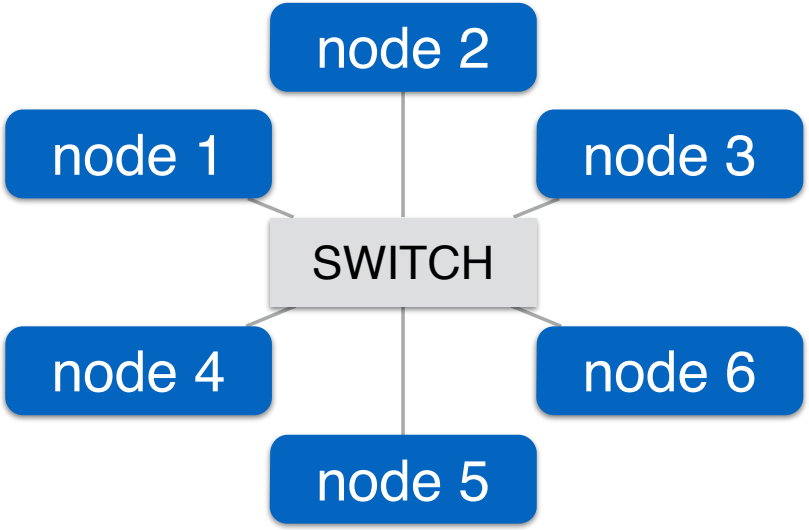
Locality-Sensitive Operators for Parallel Main-Memory Database Clusters

Wolf Rödiger, Tobias Mühlbauer, Philipp Unterbrunner*, Angelika Reiser, Alfons Kemper, Thomas Neumann

Technische Universität München, *Snowflake Computing, Inc.

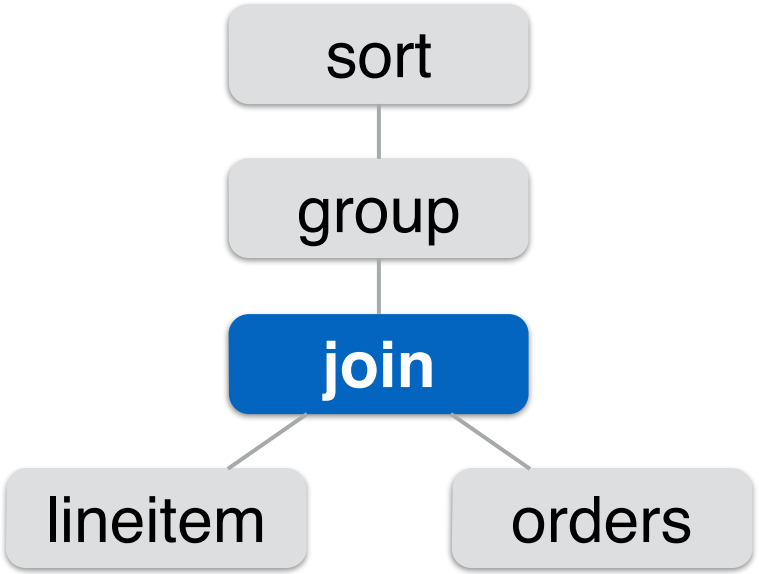
Scale Out

- **HyPer:** High-performance in-memory transaction and query processing system
- **Scale out** to process very large inputs
- Aim at **clusters** with large main memory capacity
- A server with 20 cores and 256 GB RAM costs ~$7,500



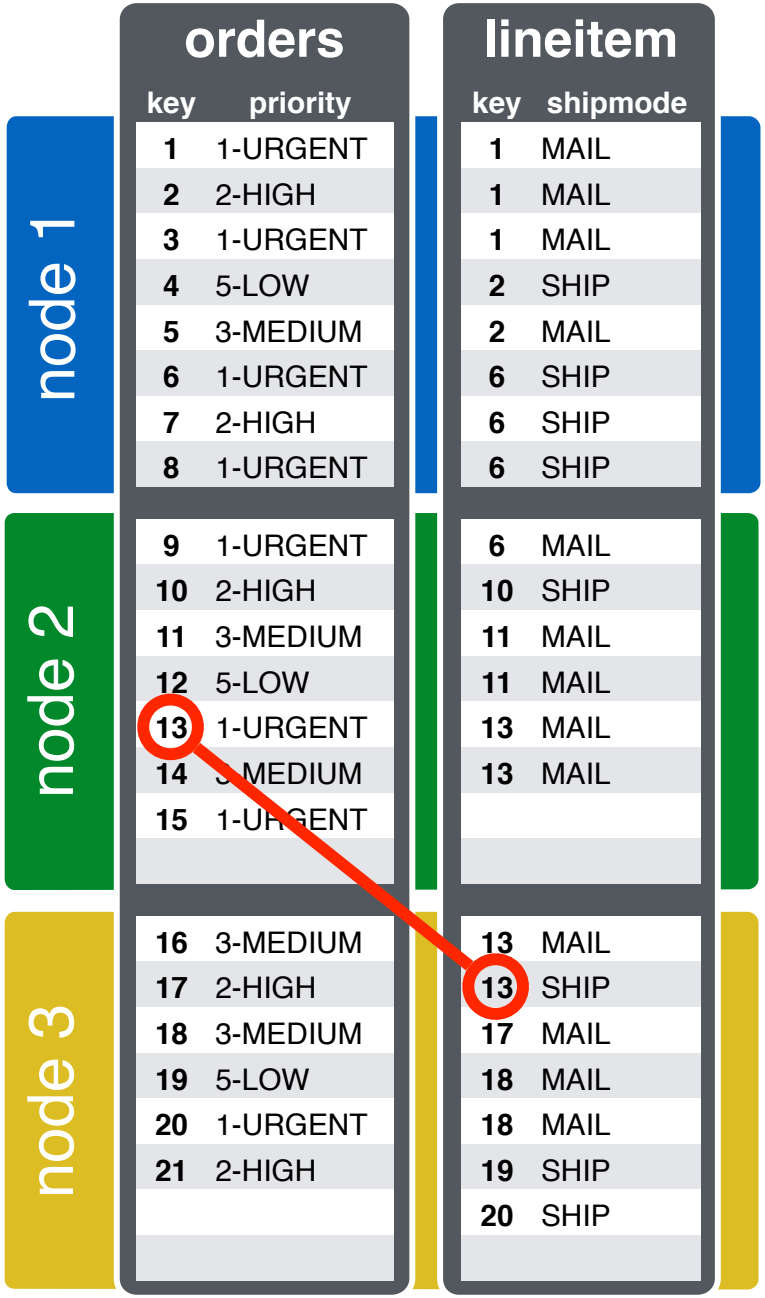
Running Example (1)

- Focus on **analytical** query processing in this talk
- TPC-H query 12 used as **running example**
- Runtime dominated by **join** orders ⋈ lineitem

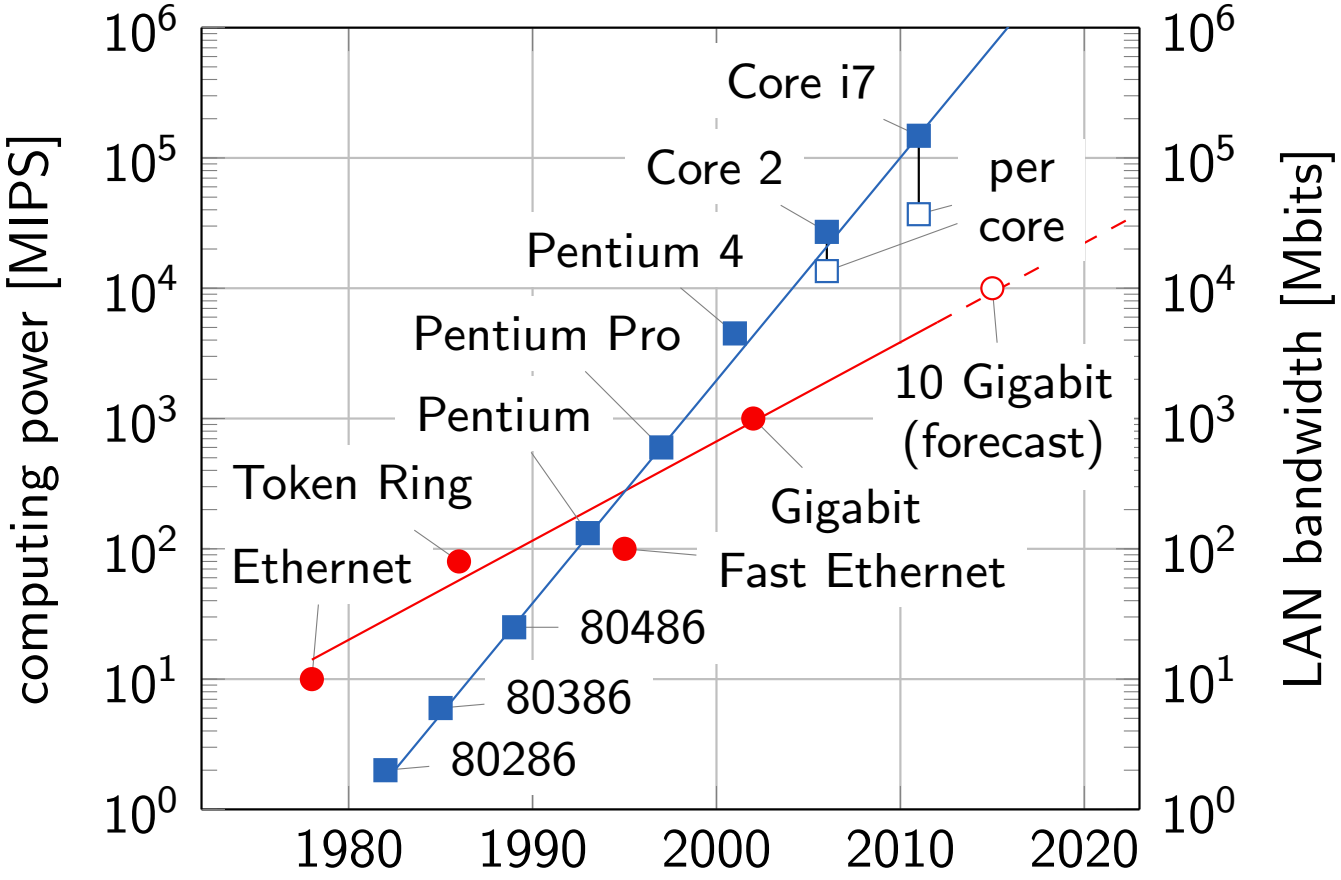


Running Example (2)

- Relations are **equally** distributed across nodes
- We make **no** assumptions on the data distribution
- Thus, tuples may join with tuples on **remote** nodes
- **Communication** over the network required



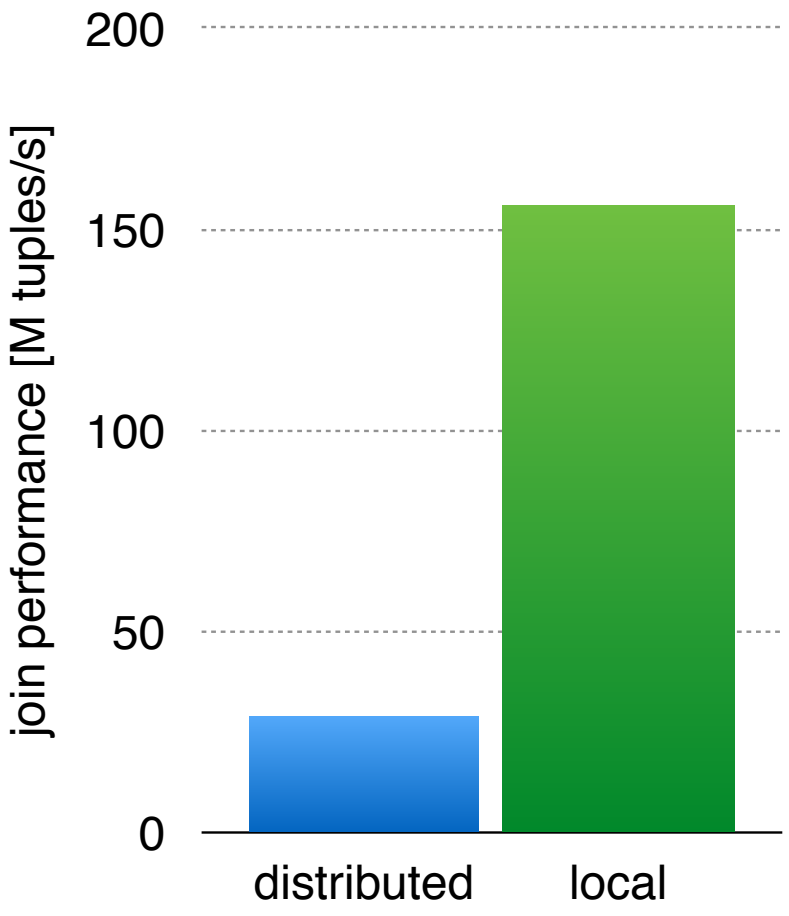
CPU vs. Network



CPU speed has grown much faster than **network bandwidth**

Scale Out: Network is the Bottleneck

- **Single node:** Performance is bound algorithmically
- **Cluster:** Network is bottleneck for query processing
- We propose a novel join algorithm called **Neo-Join**
- **Goal:**
  Increase local processing to close the performance gap



Neo-Join: Network-optimized Join

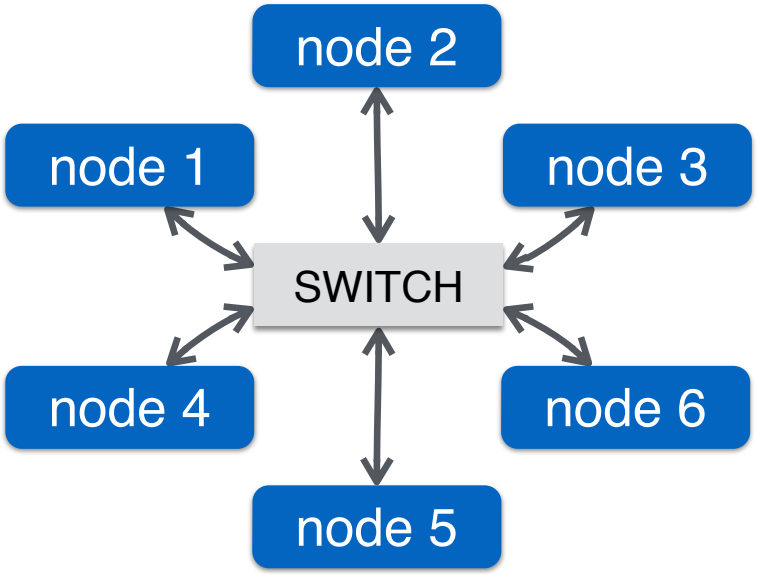
1. **Open Shop Scheduling**
   Efficient network communication
2. **Optimal Partition Assignment**
   Increase local processing
3. **Selective Broadcast**
   Handle value skew

Open Shop Scheduling

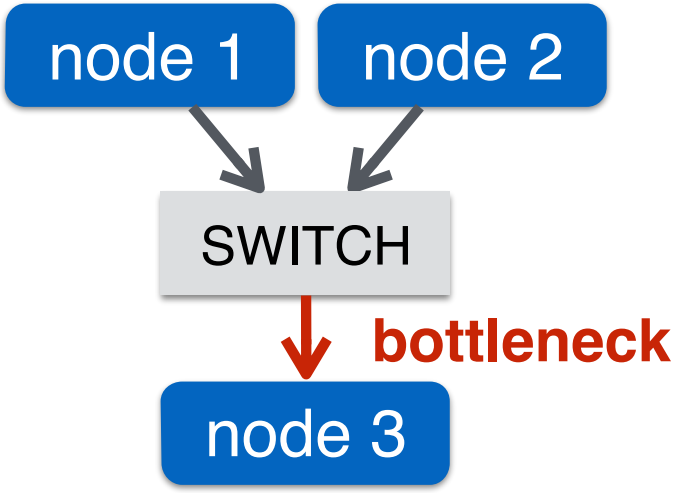
Efficient network communication

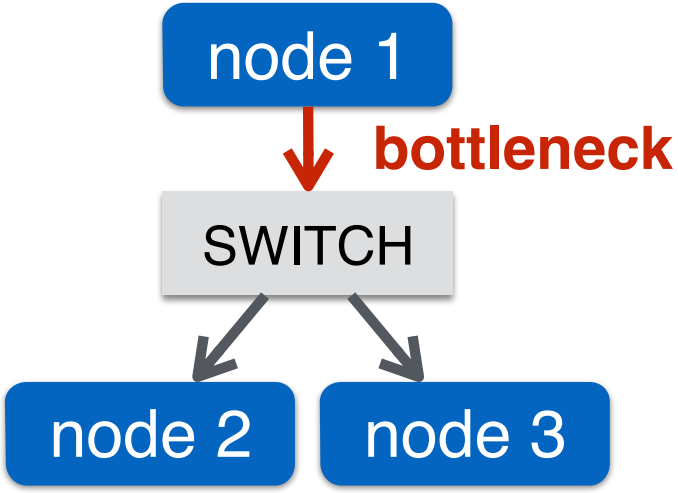
Standard Network Model

- **Star topology**
  Nodes are connected to a central switch
- **Fully switched**
  All links can be used simultaneously
- **Fully duplex**
  Nodes can both send and receive at full speed



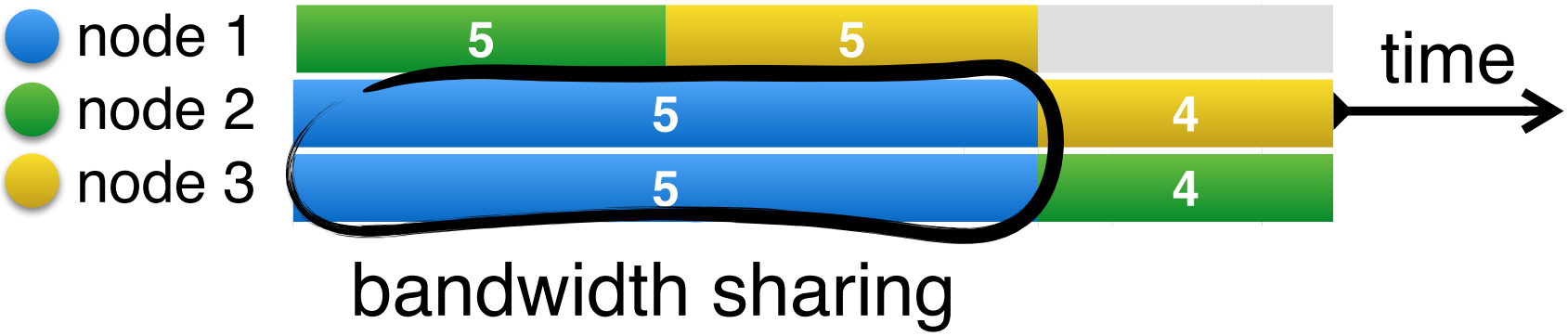
Bandwidth Sharing





- Simultaneous use of a single link creates a **bottleneck**
- **Reduces bandwidth** by at least a factor of **2**

Naïve Schedule

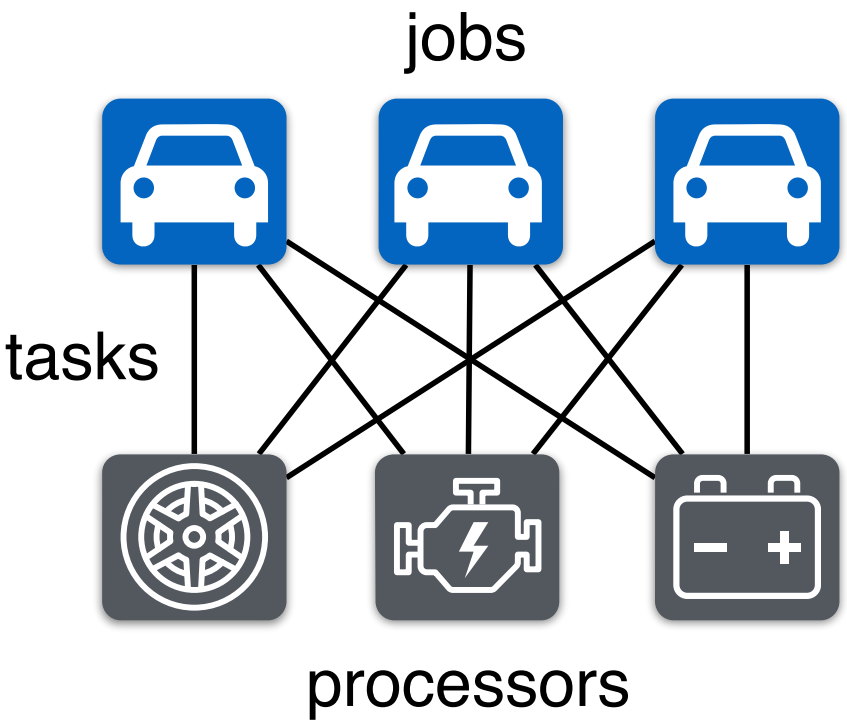


- Node 2 and 3 send to node 1 **at the same time**
- Bandwidth sharing increases **network duration** significantly

Open Shop Scheduling (1)

Avoiding bandwidth sharing translates to **open shop scheduling:**

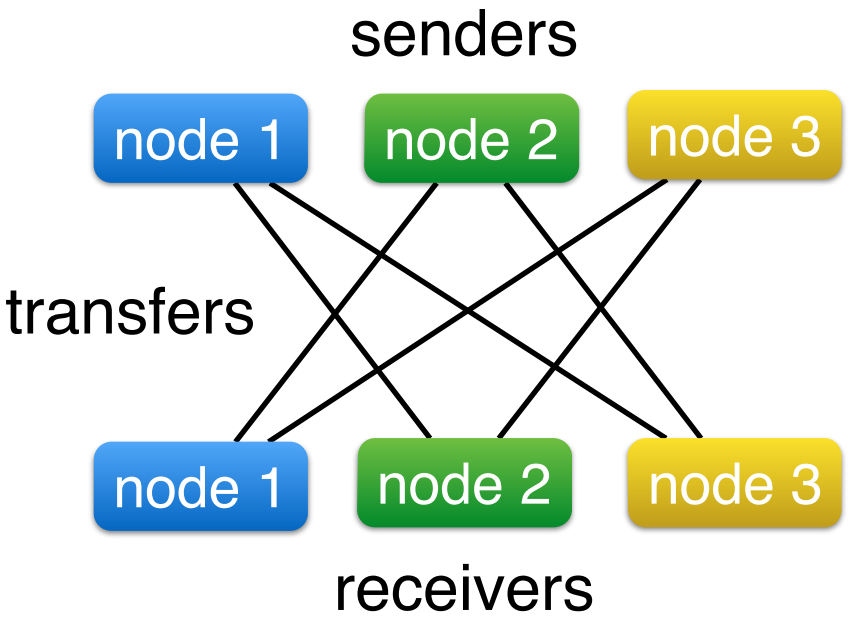
- A **job** consists of one **task** per **processor**
- A processor can perform at most **one** task at a time
- At most **one** task of a job can be processed at a time



Open Shop Scheduling (2)

Avoiding bandwidth sharing translates to **open shop scheduling:**

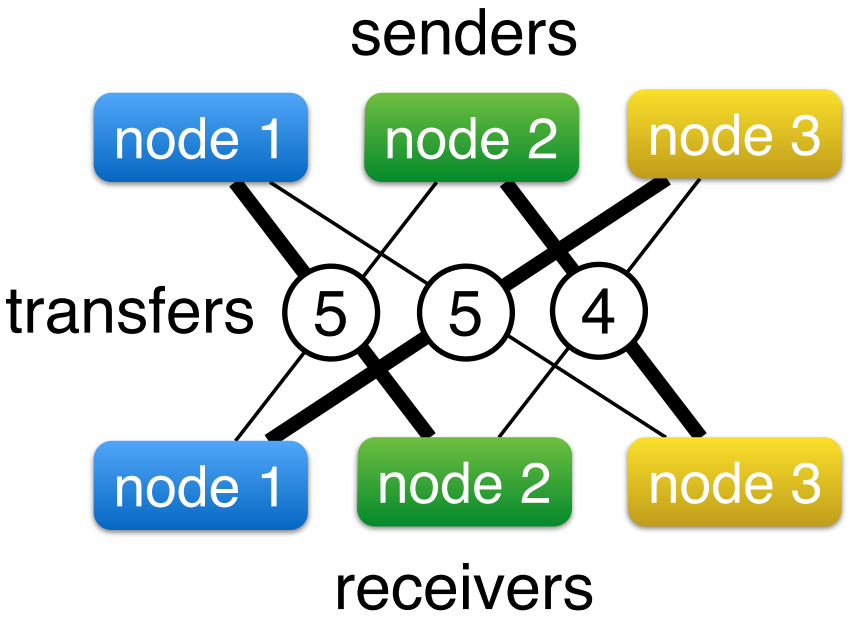
- A **sender** has one **transfer** per **receiver**
- A receiver should receive at most **one** transfer at a time
- A sender should send at most **one** transfer at a time



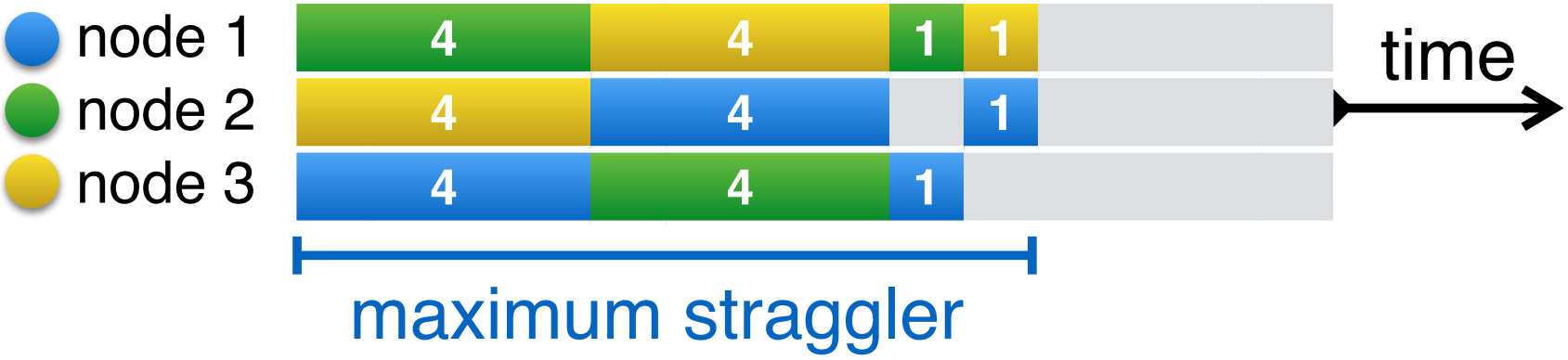
Open Shop Scheduling (3)

Compute optimal schedule:

- **Edge weights** represent total transfer duration
- Scheduler repeatedly finds **perfect matchings**
- Each matching specifies one communication **phase**
- Transfers in a phase will **never** share bandwidth



Optimal Schedule



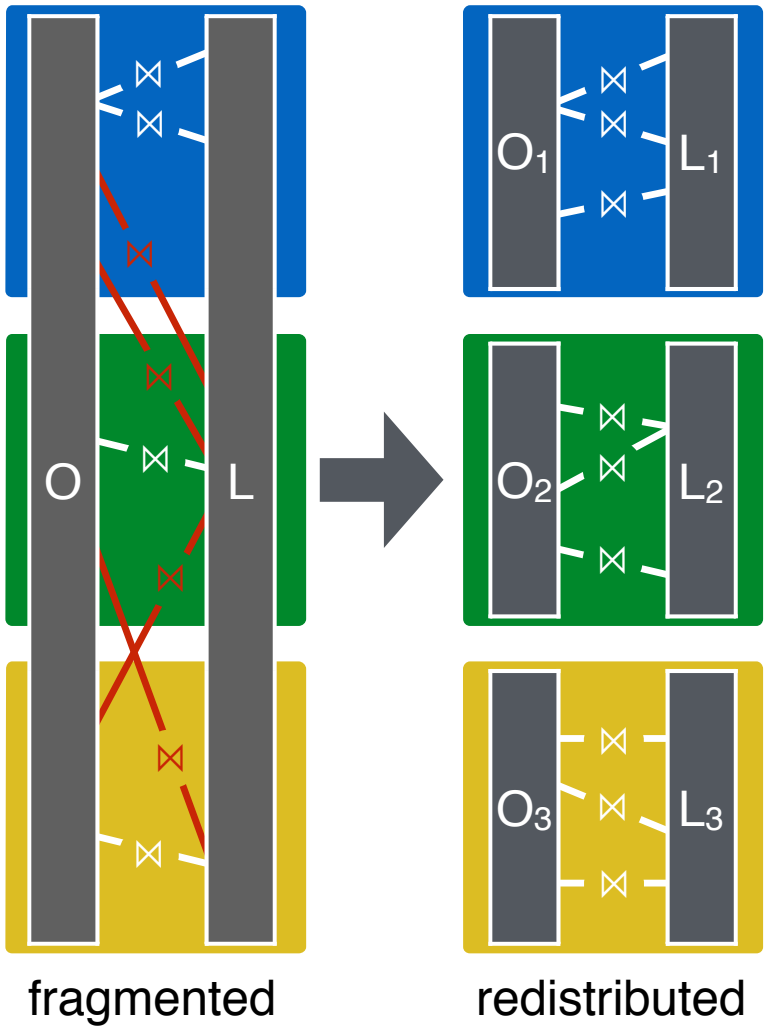
- Open shop schedule achieves minimal **network duration**
- Schedule duration determined by **maximum straggler**

Optimal Partition Assignment

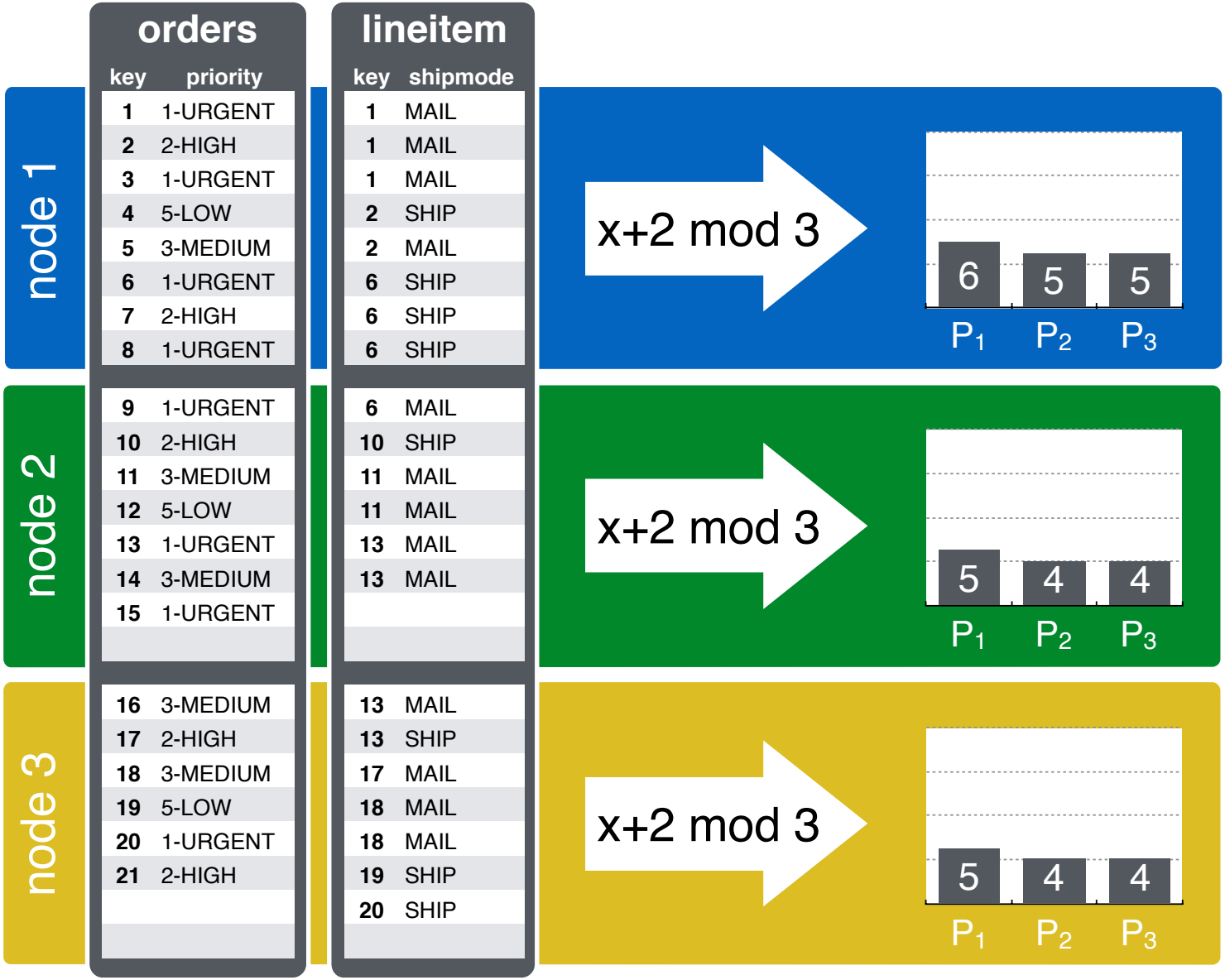
Minimize network duration for distributed joins

Distributed Join

- Tuples may join with tuples on **remote nodes**
- Repartition and redistribute **both relations** for local join
- Tuples will join only with the **corresponding partition**
- Using hash, range, radix, or other **partitioning** scheme
- **In any case:** Decide how to **assign** partitions to nodes



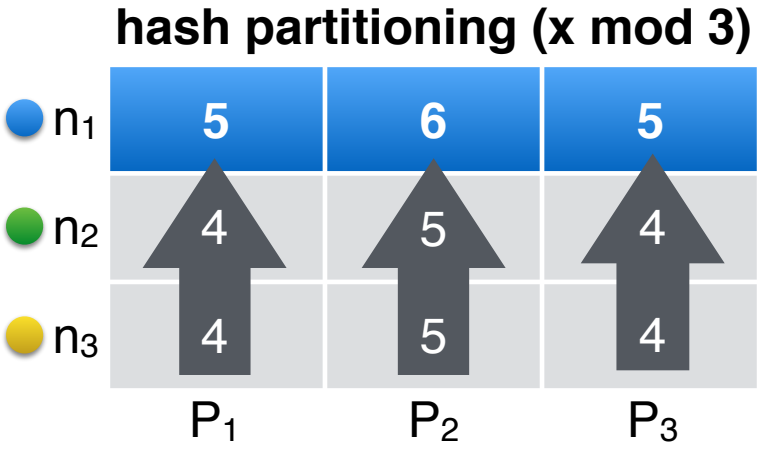
Running Example: Hash Partitioning

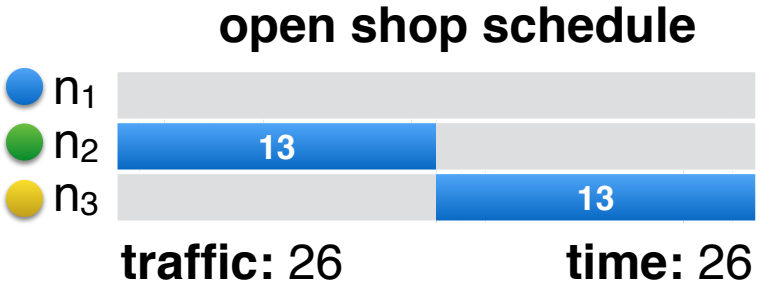


Assign Partitions to Nodes (1)

Option 1: Minimize network traffic

- Assign partition to node that owns its **largest part**
- Only the **small fragments** of a partition sent over the network
- Schedule with minimal network traffic may have **high duration**

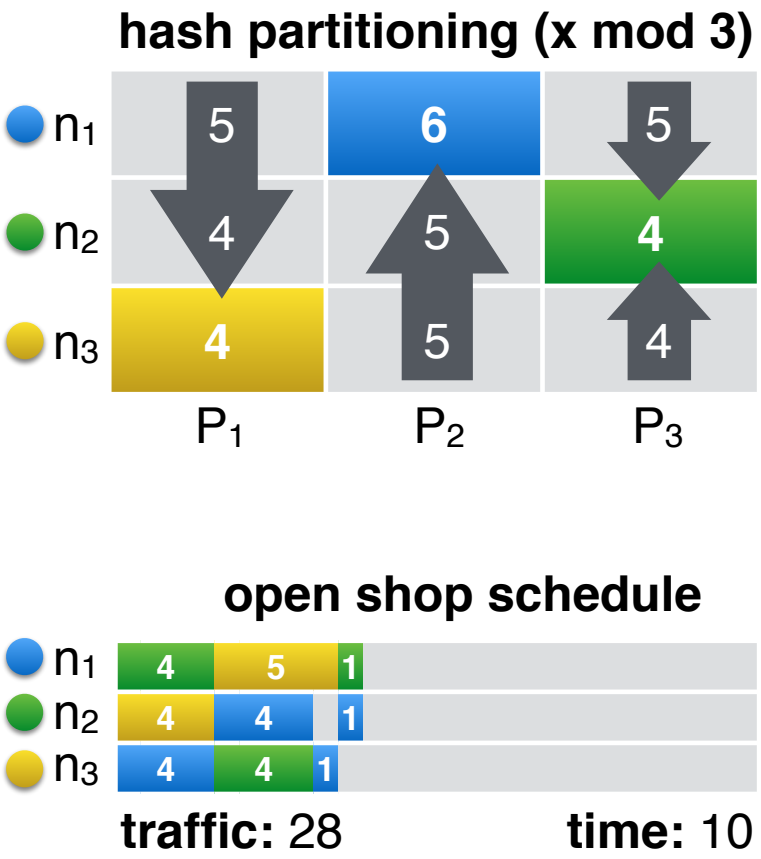




Assign Partitions to Nodes (2)

Option 2: Minimize response time:

- **Query response time** is time from request to result
- Query response time dominated by **network duration**
- To minimize network duration, minimize **maximum straggler**



Minimize Maximum Straggler

- Formalized as mixed-integer **linear program**
- Shown to be **NP-hard** (see paper for proof sketch)
- In practice **fast enough** using CPLEX or Gurobi (< 0.5 % overhead for 32 nodes, 200 M tuples each)
- Partition assignment can optimize **any partitioning**

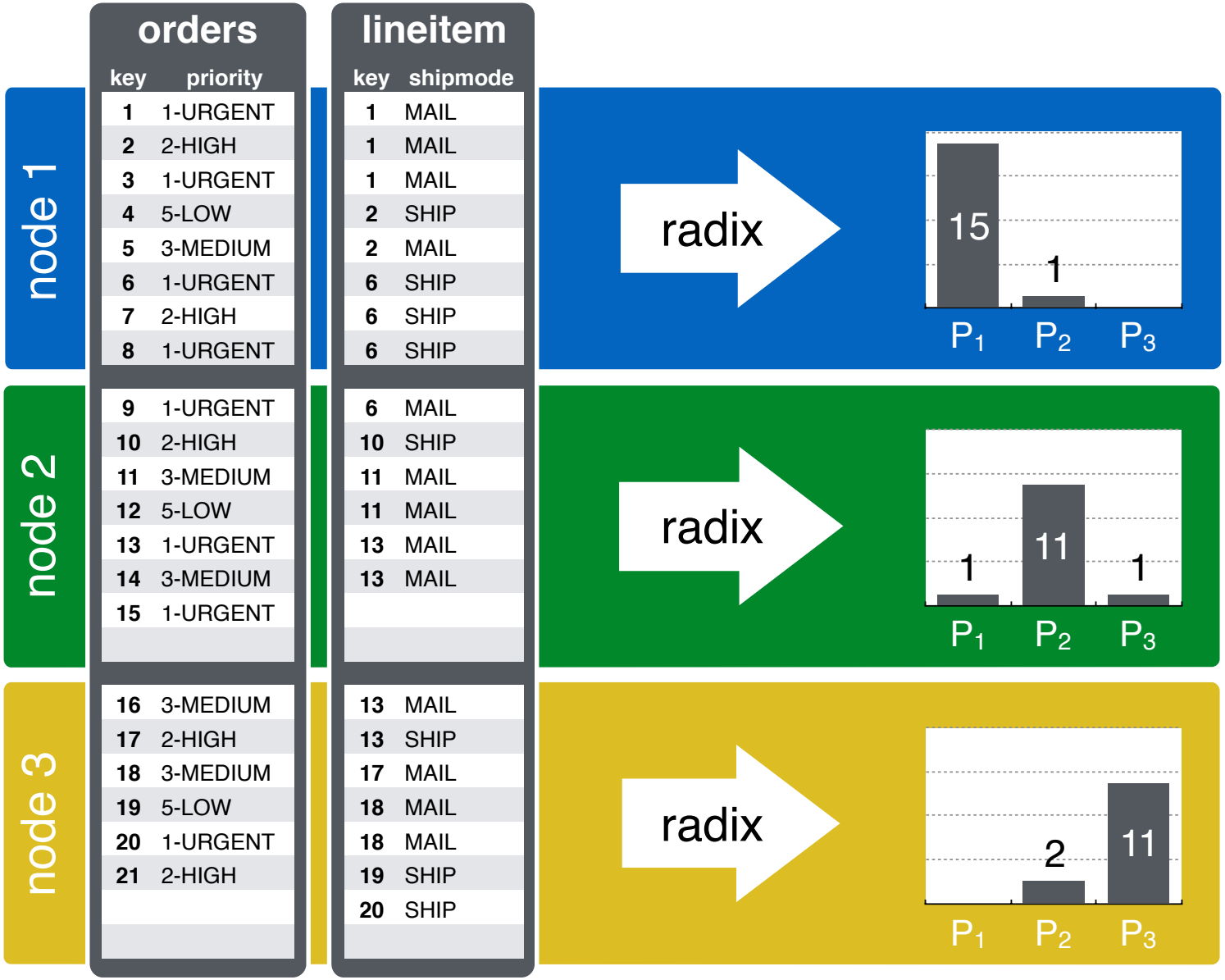
minimize w, subject to

$$w \geq \sum_{j=0}^{p-1} h_{ij}(1 - x_{ij}) \qquad 0 \leq i < n$$

$$w \geq \sum_{j=0}^{p-1} \left(x_{ij} \sum_{k=0, i \neq k}^{n-1} h_{kj} \right) \qquad 0 \leq i < n$$

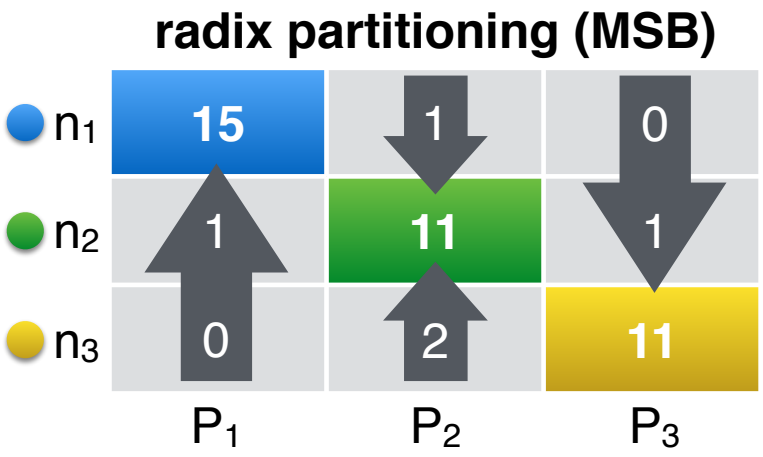
$$1 = \sum_{i=0}^{n-1} x_{ij} \qquad 0 \leq j < p$$

Running Example: Locality



Locality

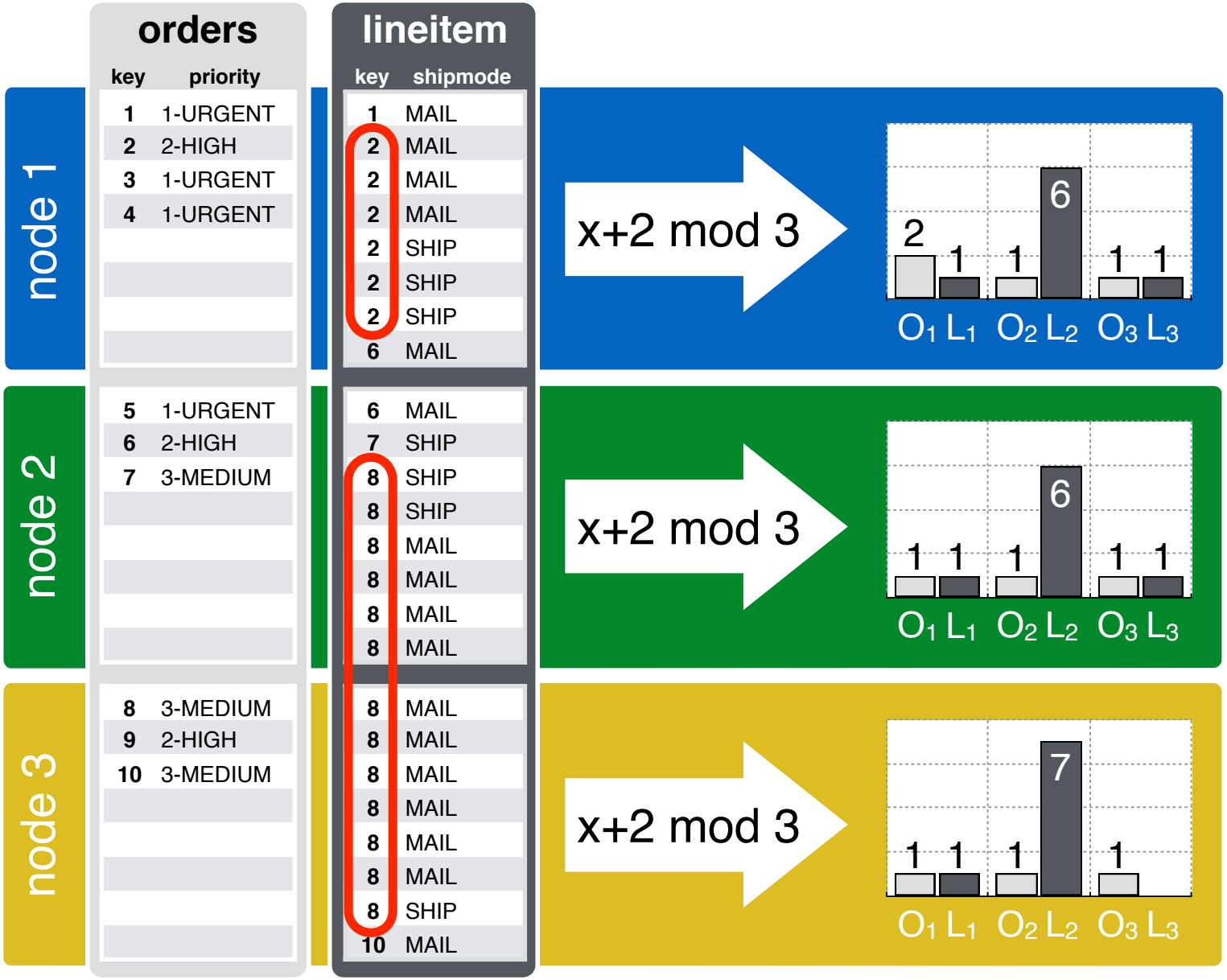
- Running example exhibits **time-of-creation** clustering
- **Radix repartitioning** on most significant bits retains locality
- Partition assignment can **exploit locality**
- Significantly reduces **query response time**



Selective Broadcast

Handle value skew

Running Example: Skew



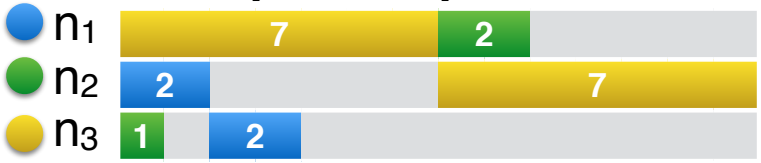
Skew

- **Skewed** partition P_2 has to be assigned, e.g., to node 3
- Node 3 will receive **much more** than its fair share
- May balance skewed partitions by creating **more partitions**
- **However:** More expensive and **high skew** is still a problem

hash partitioning (mod 3)

	P_1	P_2	P_3
n_1	3	7	2
n_2	2	7	2
n_3	2	8	1

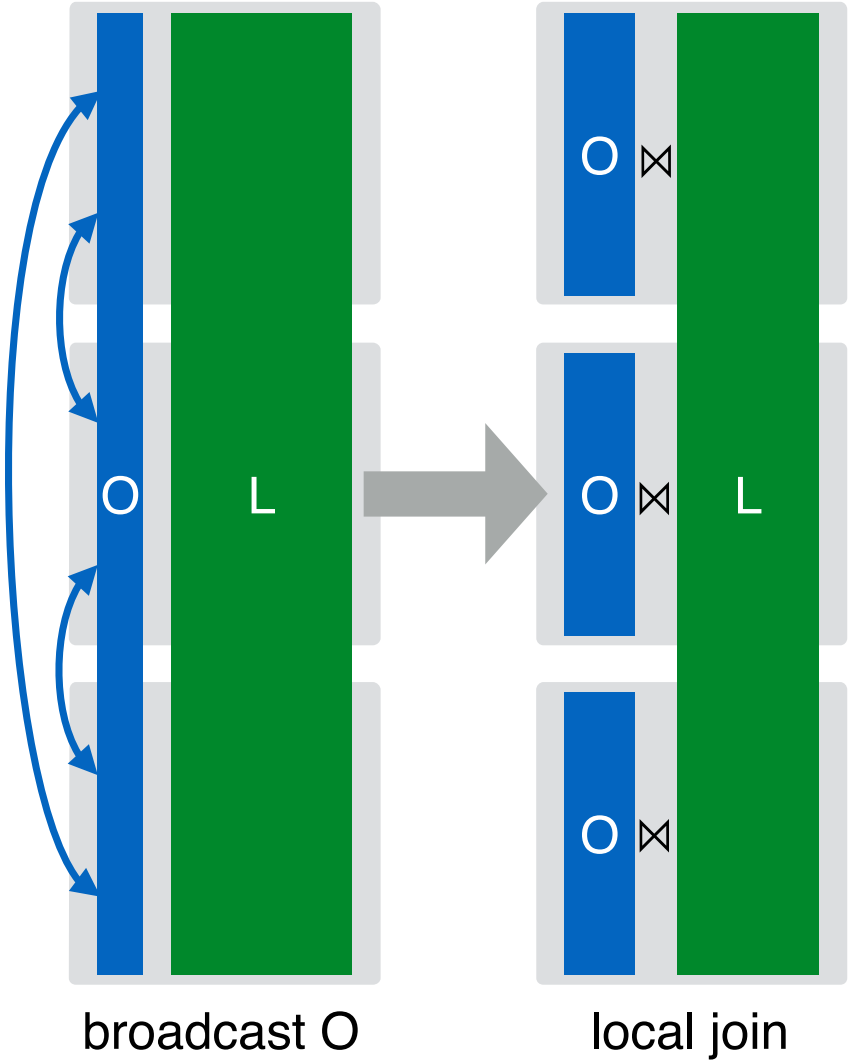
open shop schedule



traffic: 21 **time:** 14

Broadcast

- **Alternative** to data repartitioning
- **Replicate** the smaller relation between all nodes
- Larger relation **remains fragmented** across nodes



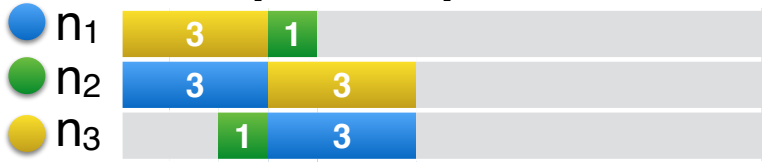
Selective Broadcast

- Decide **per partition** whether to assign or broadcast
- **Broadcast** orders for P_2, let line items remain fragmented
- **Assign** the other partitions taking locality into account
- Improves performance for high **skew** and many **duplicates**

hash partitioning (mod 3)

	O_1	L_1	O_2	L_2	O_3	L_3
n_1	2	1	1	6	1	1
n_2	1	1	1	6	1	1
n_3	1	1	1	5	2	2

open shop schedule



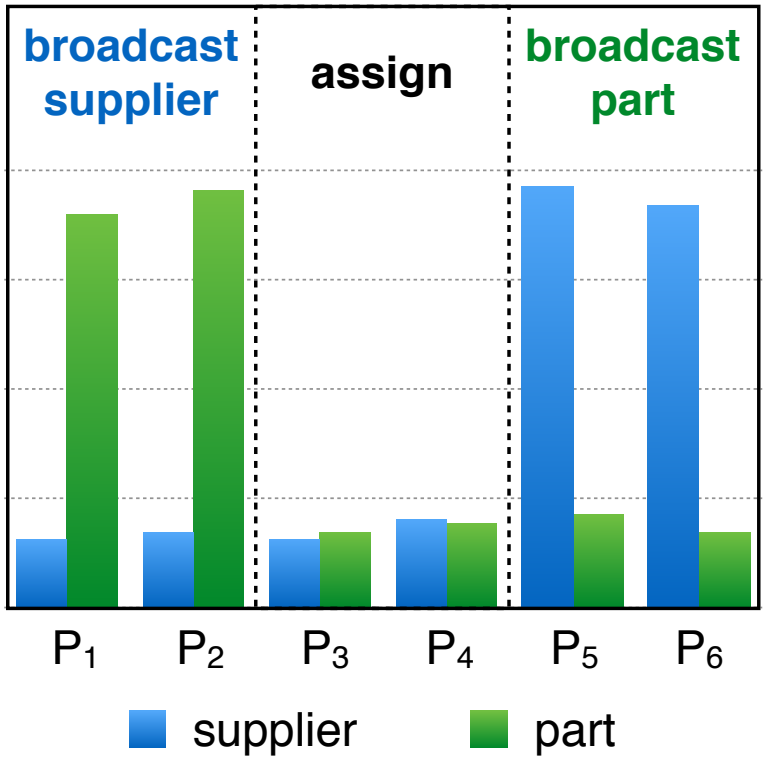
traffic: 14 **time:** 6

Role Reversal

- Selective broadcast allows for **role reversal**
- Broadcast different partitions by **different relations**

Example:

- **Large suppliers** produce a large variety of parts
- **Important parts** available from many suppliers



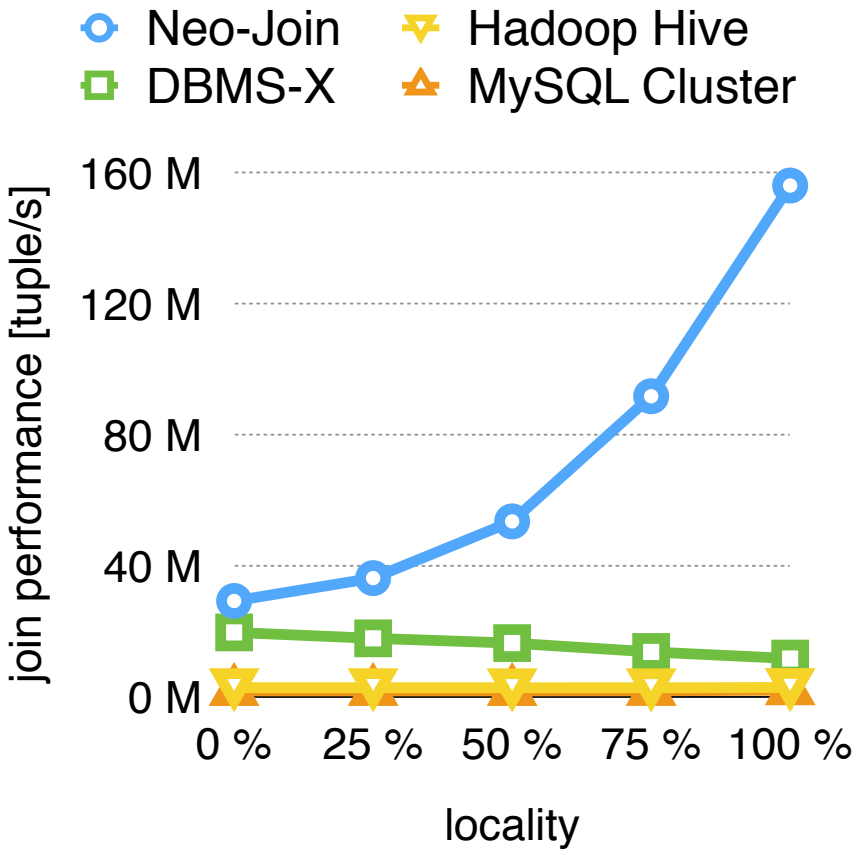
Evaluation

Experimental Setup

- Cluster of 4 nodes
- Core i7, 4 cores, 3.4 GHz, 32 GB RAM
- Gigabit Ethernet
- Tuples consist of 64 bit key, 64 bit payload

Locality

- Vary **locality** from **0 %** (uniform distribution) to **100 %** (range partitioning)
- Neo-Join improves **join performance** from 29 M to 156 M tuples/s (> 500 %)
- 3 nodes, 600 M tuples



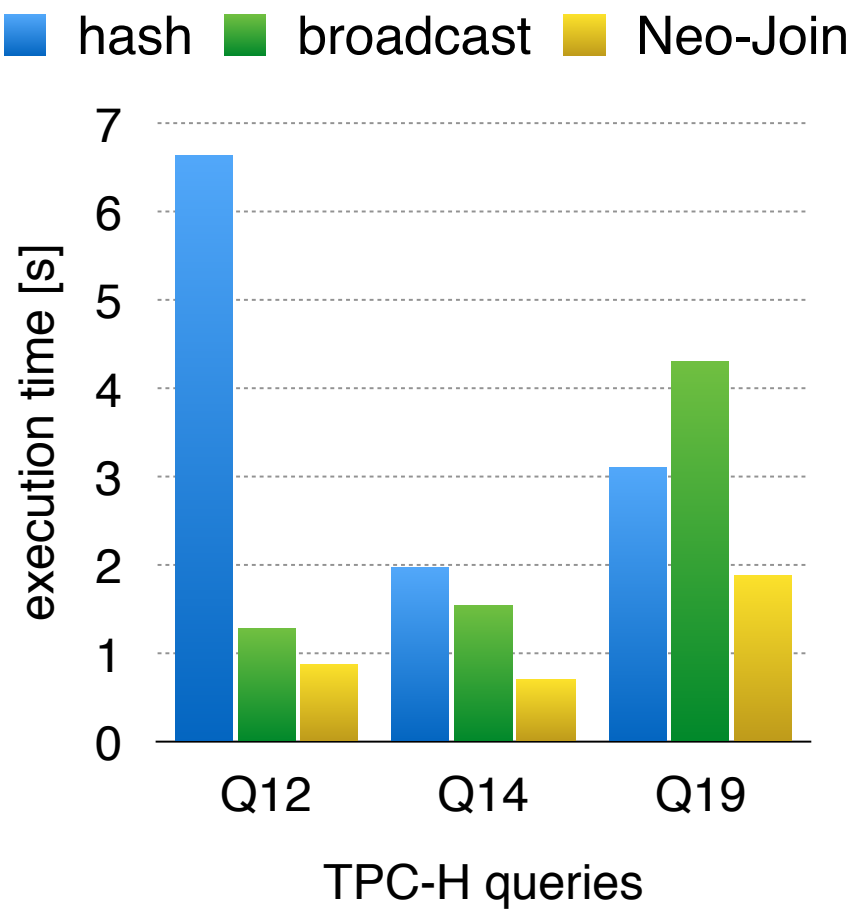
Skew

- **Zipfian distribution** models realistic data skew
- Using **more partitions** alleviates the problem
- Selective broadcast actually **improves** performance for skewed inputs
- 4 nodes, 400 M tuples

	Zipf factor s				
partitions	0.00	0.25	0.50	0.75	1.00
16	27 s	24 s	23 s	**29s**	**44s**
512	23 s	23 s	23 s	23 s	**33s**
16 (SB)	24 s	24 s	23 s	**20s**	**10s**

TPC-H Results (scale factor 100)

- Results for three selected **TPC-H** queries
- **Broadcast** outperforms **hash** for large relation size differences
- Neo-Join always performs better due to **selective broadcast** and **locality**
- 4 nodes, scale factor 100



Summary

Motivation:

- **Scale out** to handle very large inputs
- **Network** is the bottleneck
- Thus, **reduce** network duration

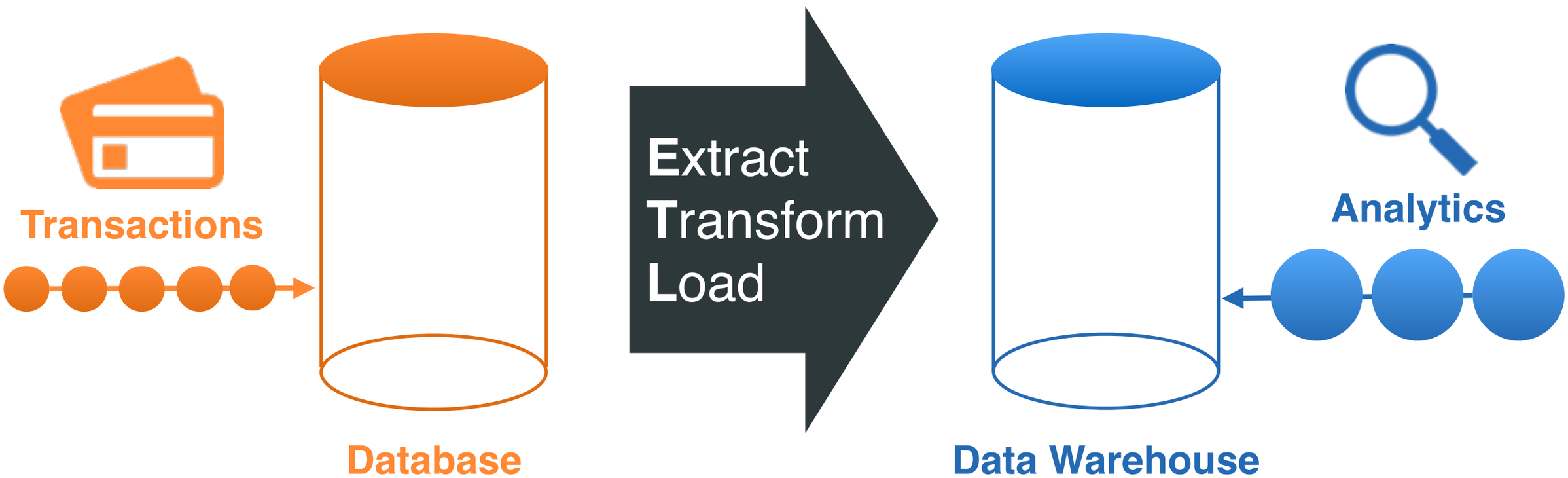
Contributions:

- Maximize bandwidth usage with **Open Shop Scheduling**
- Exploit locality with **Optimal Partition Assignment**
- Handle skewed inputs with **Selective Broadcast**

High-Speed Query Processing over High-Speed Networks

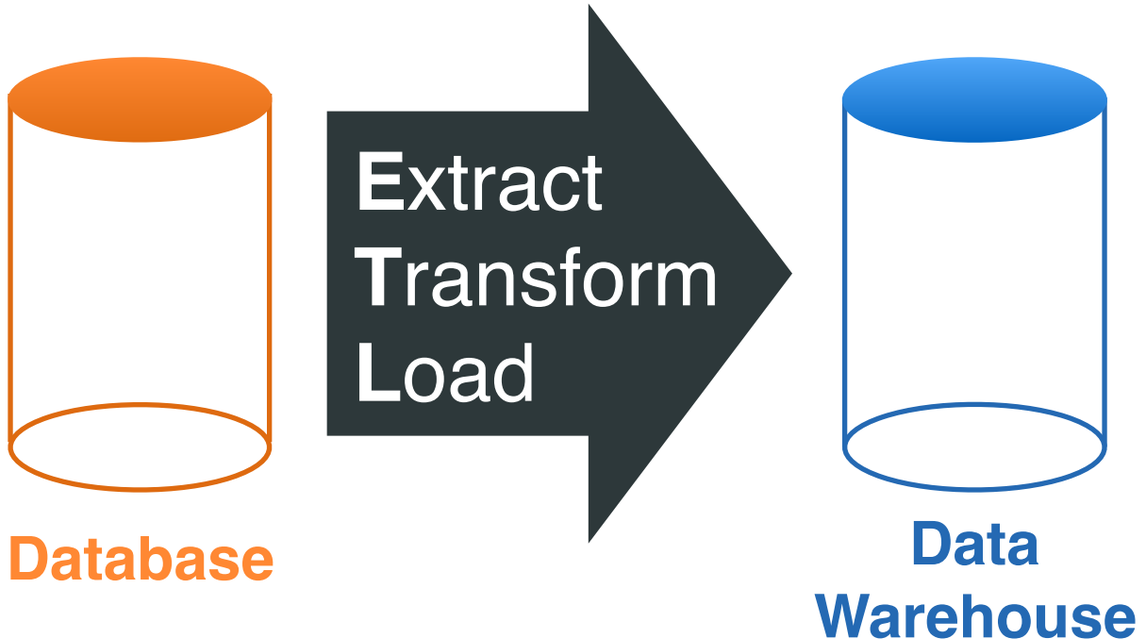
Wolf Rödiger, Tobias Mühlbauer, Alfons Kemper, Thomas Neumann

Traditional Data Warehouse



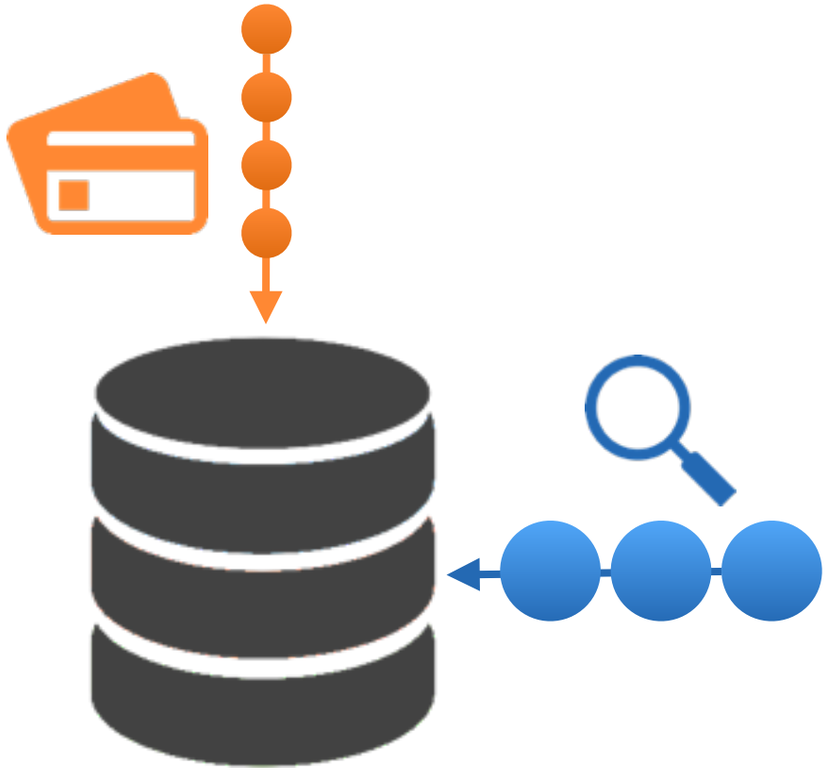
Traditional Data Warehouse

- Isolate business-critical **transactions** from analytical **queries**
- **ETL process** to update the data warehouse
- Periodic refresh leads to **data staleness**



HyPer

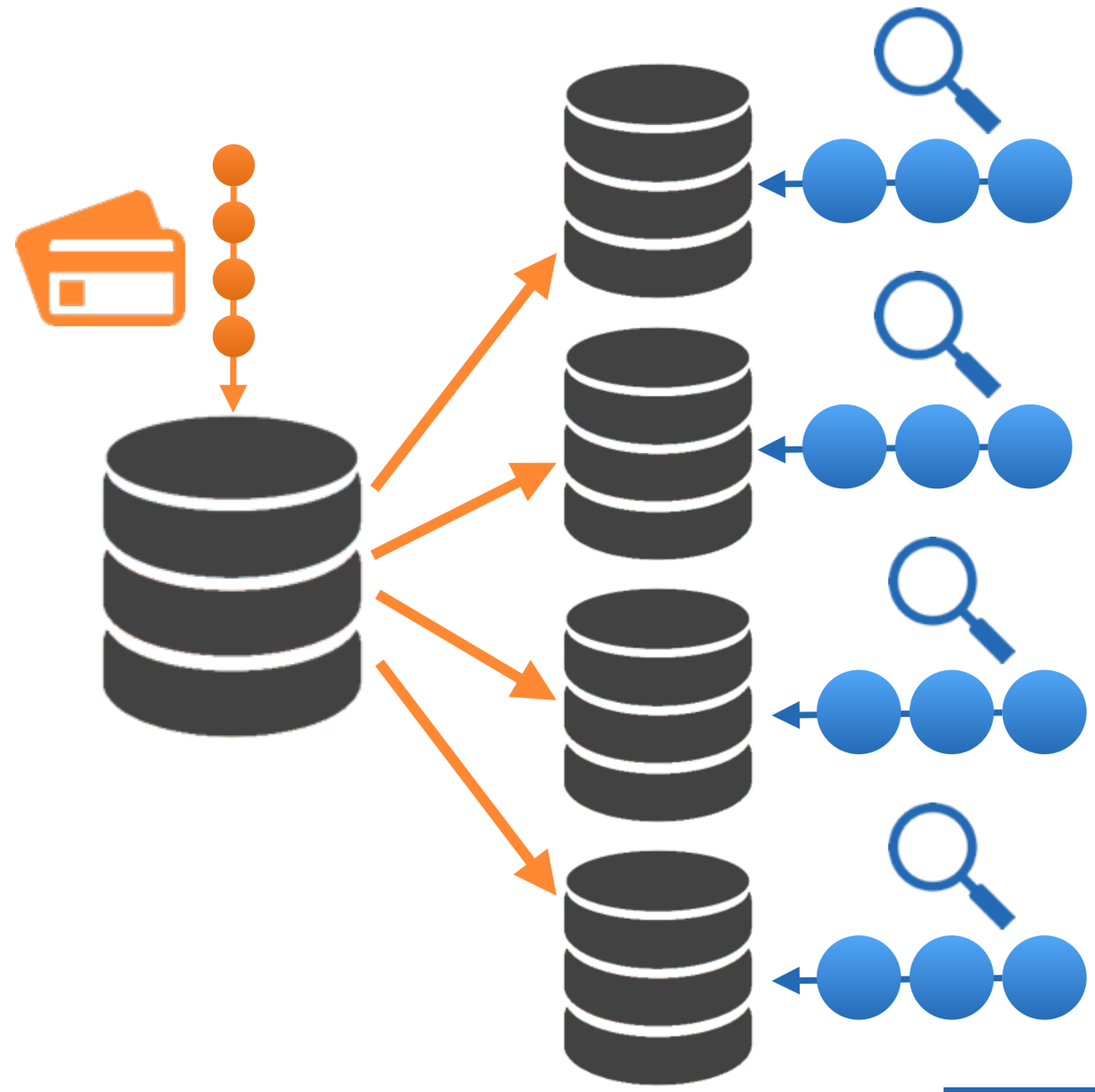
- **Analytics**
  Excellent response times
- **Transactions**
  100k TPC-C transaction/s
- **Both workloads** on the same state in one system
- Code generation, MVCC



Scale the **HyPer** main-memory database system to a **cluster** of machines

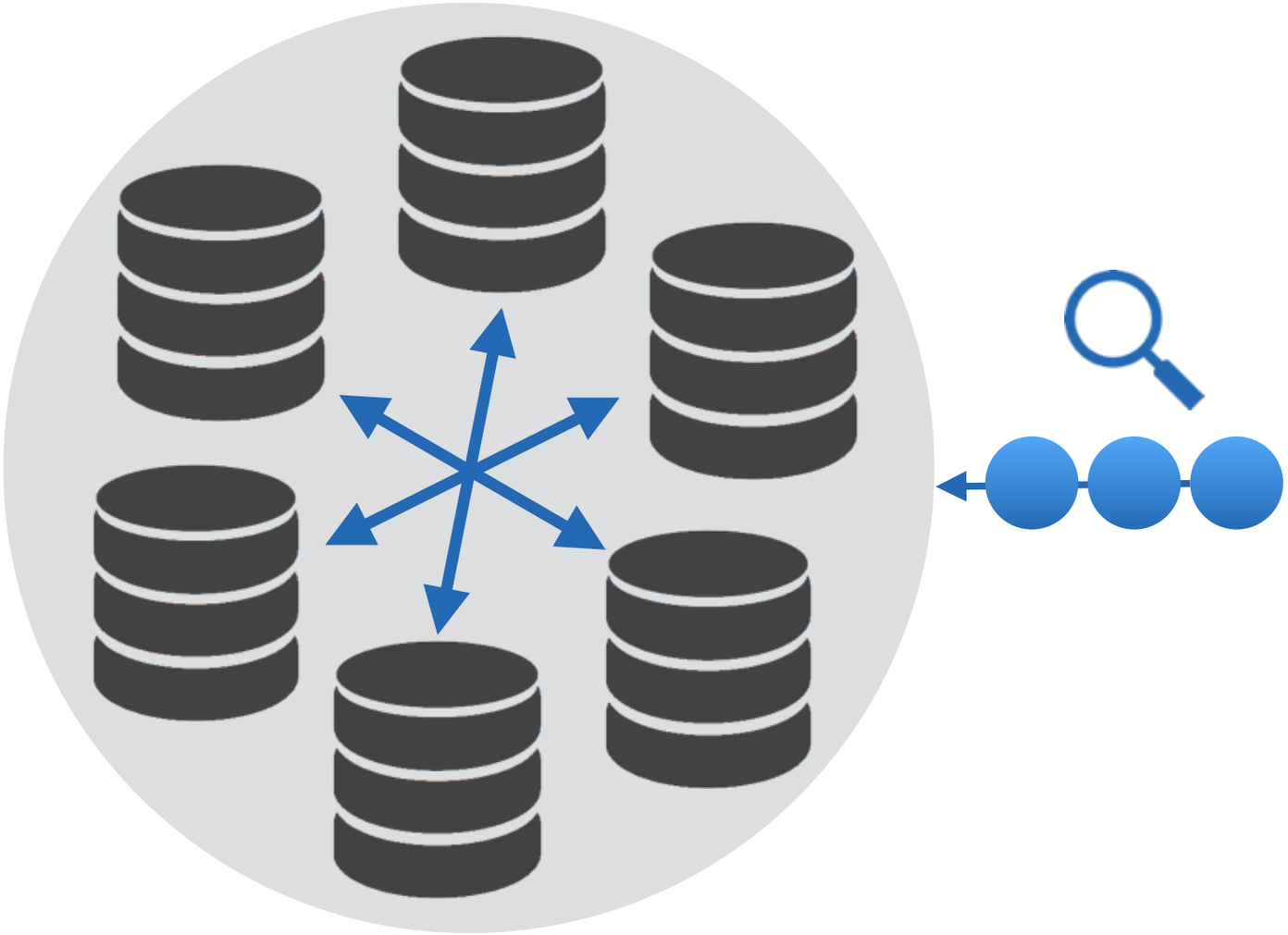
Full Replication

- **Replicate** the data from a primary server
- **Improved** query throughput
- **Same** main-memory, **same** response times as a single server

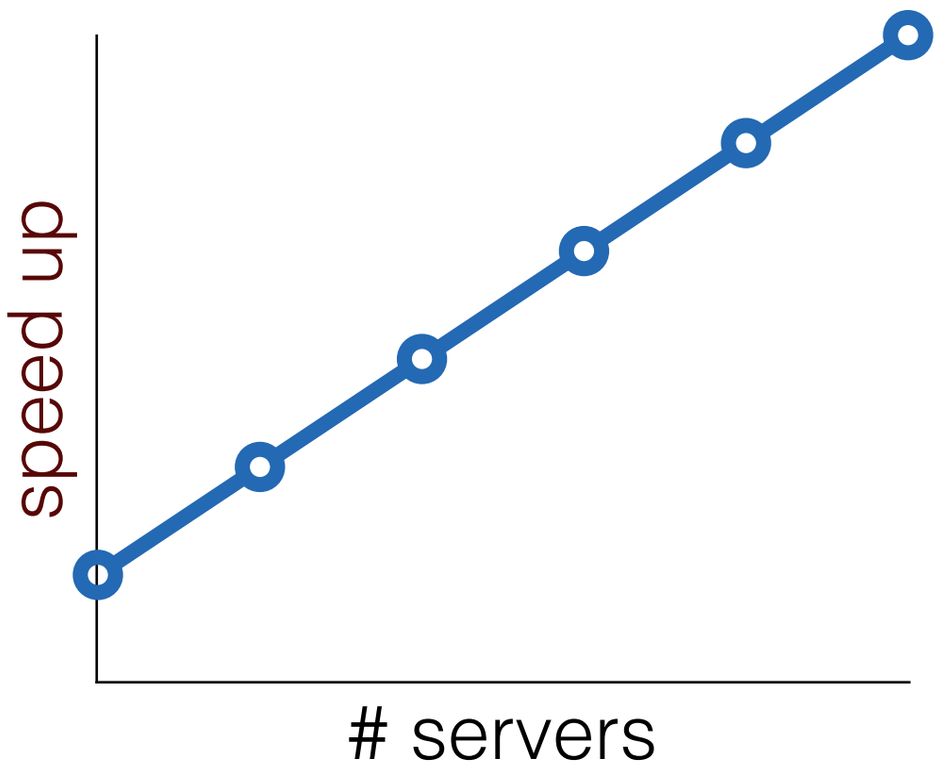


Horizontal Partitioning

- **Partition** the data across servers
- **Increases** main-memory capacity
- **But can we also speed up query processing?**

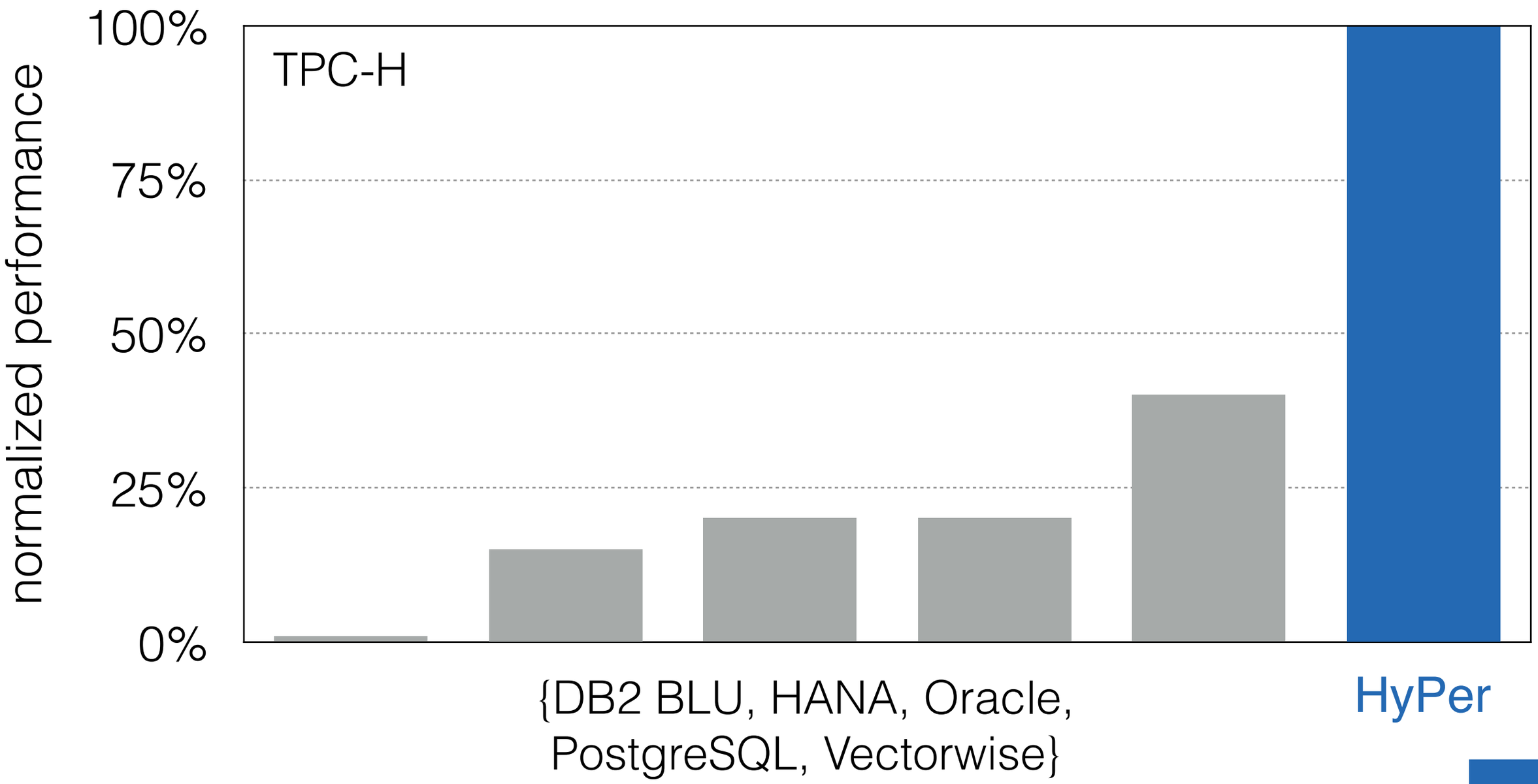


Speed-Up? Easy!



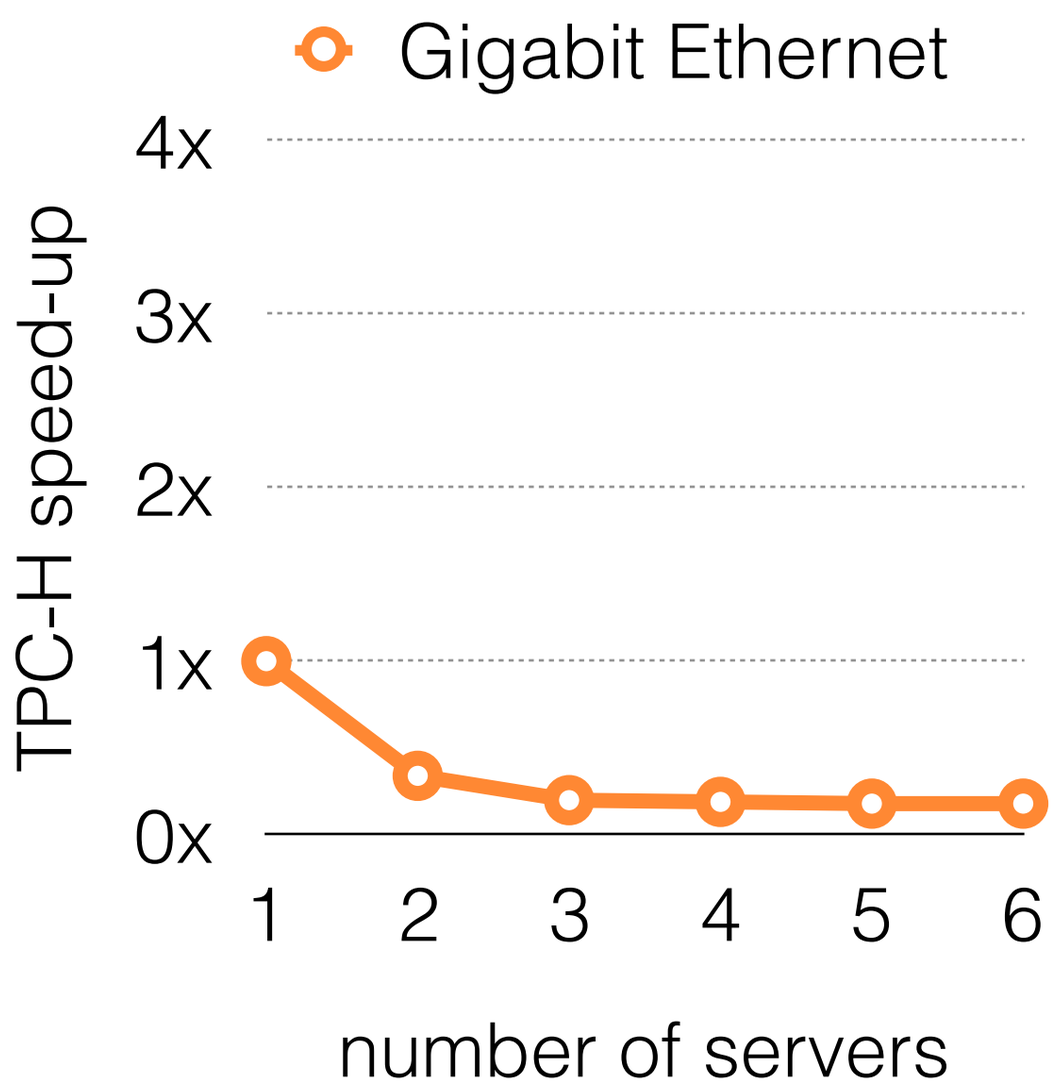
… as long as the system is **slow** on **one server.**

Unfortunately, *HyPer is Fast*



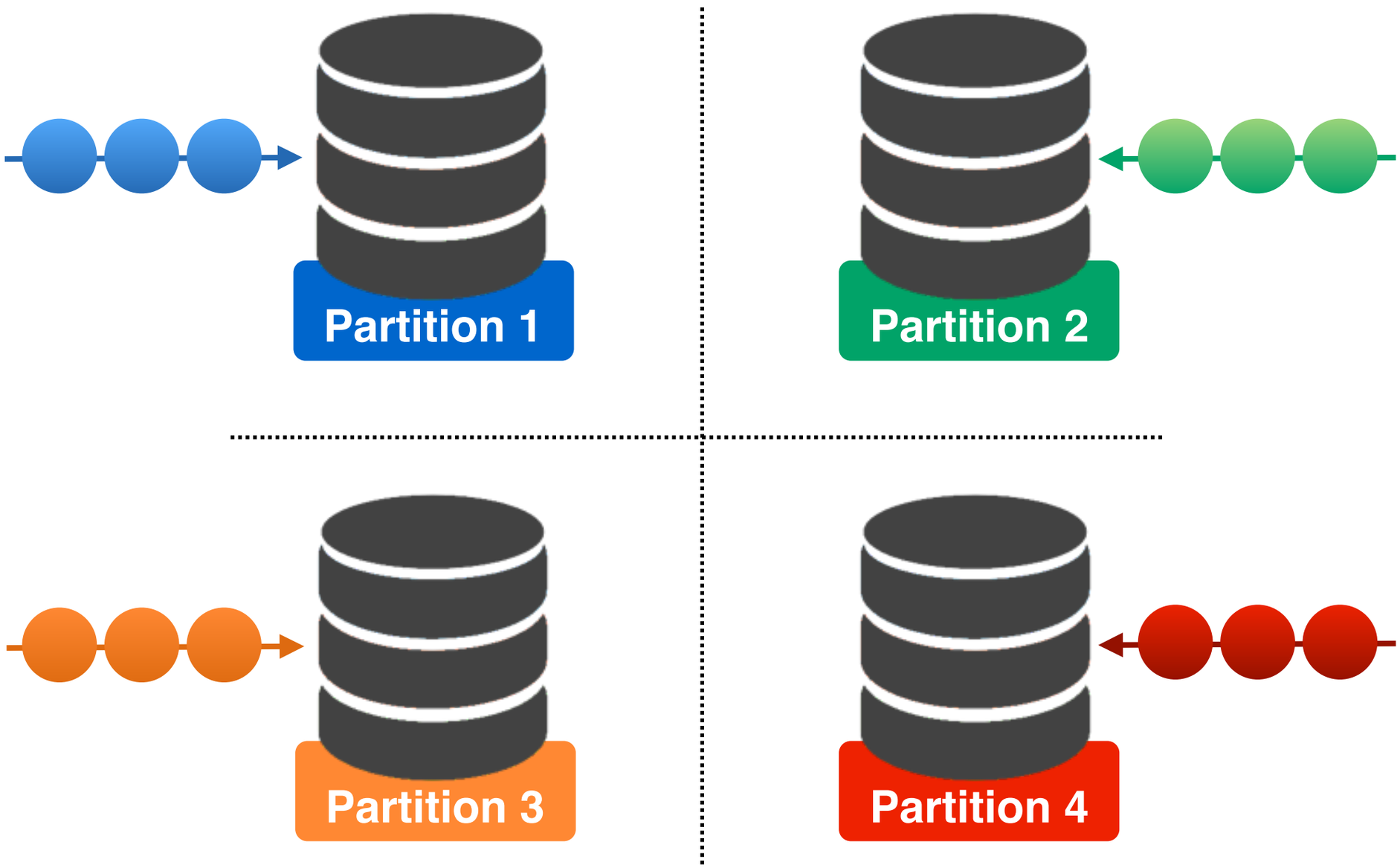
Negative Speed-Up

- Queries **shuffle data** for joins and aggregations
- **Low bandwidth** is main bottleneck
- More servers = **less performance**



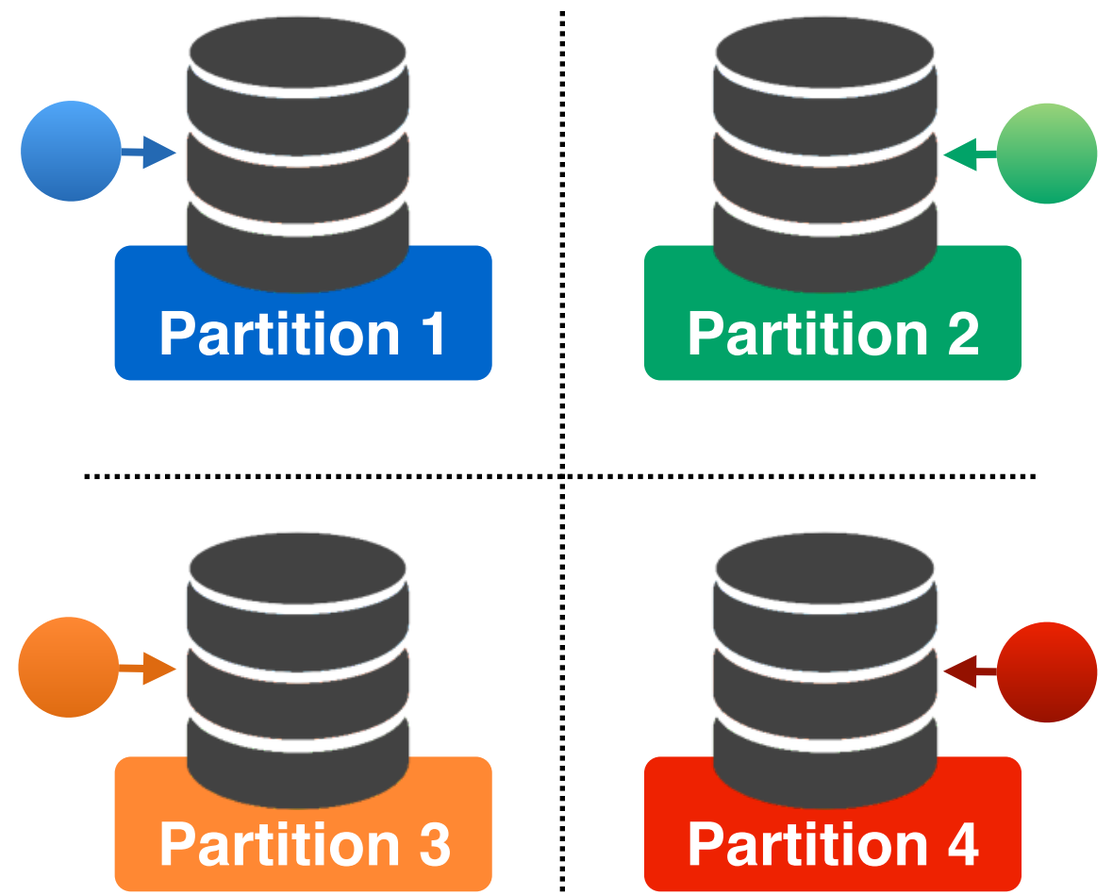
Scale HyPer to a cluster and it should be **fast**

Can’t we just avoid communication?



Can’t we just avoid communication?

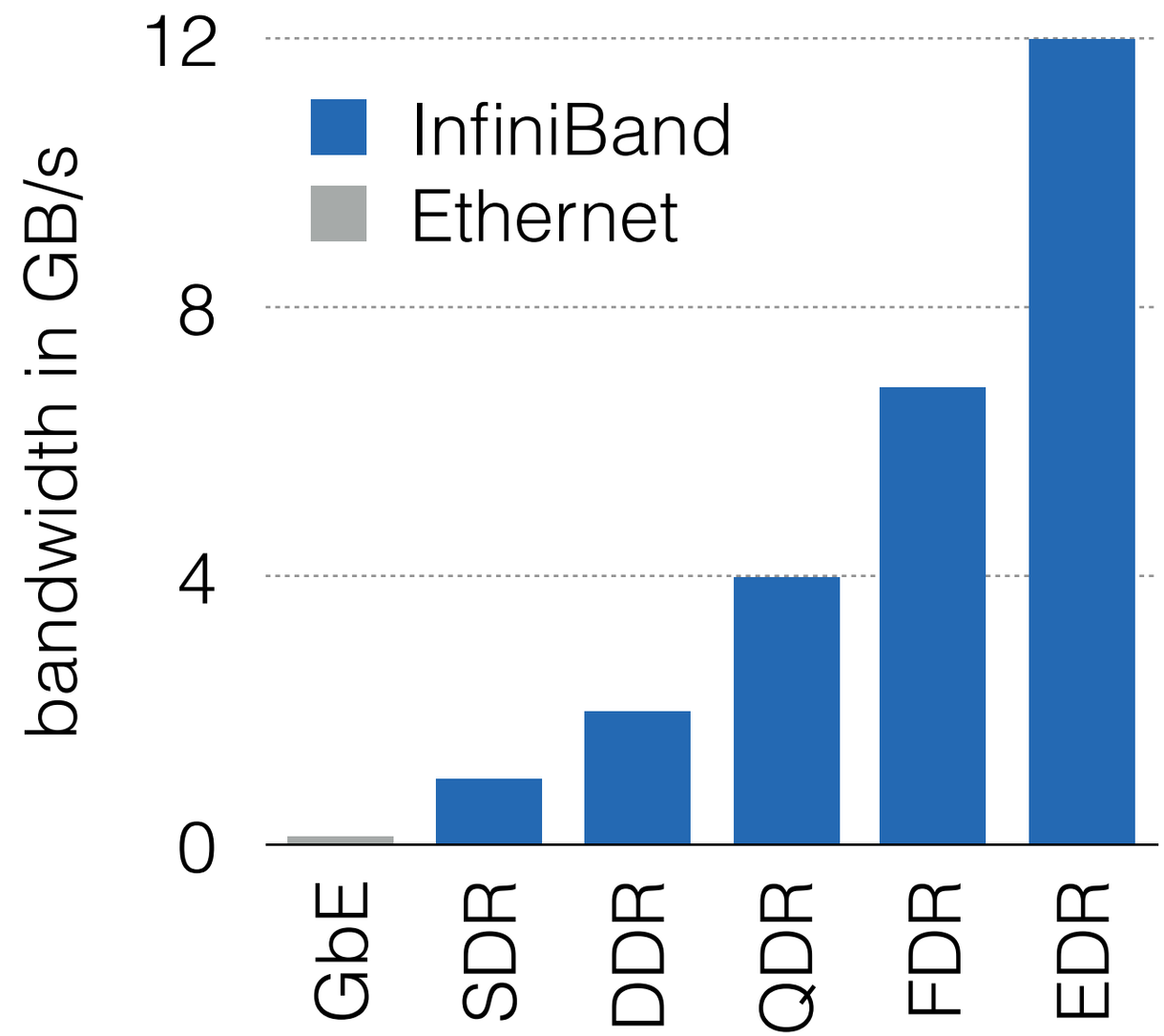
- **Partition** the data (H-Store/VoltDB do this for transactions)
- **Partition-crossing** queries problematic
- Partitioning depends on the **workload**



Can’t we just use faster network hardware?

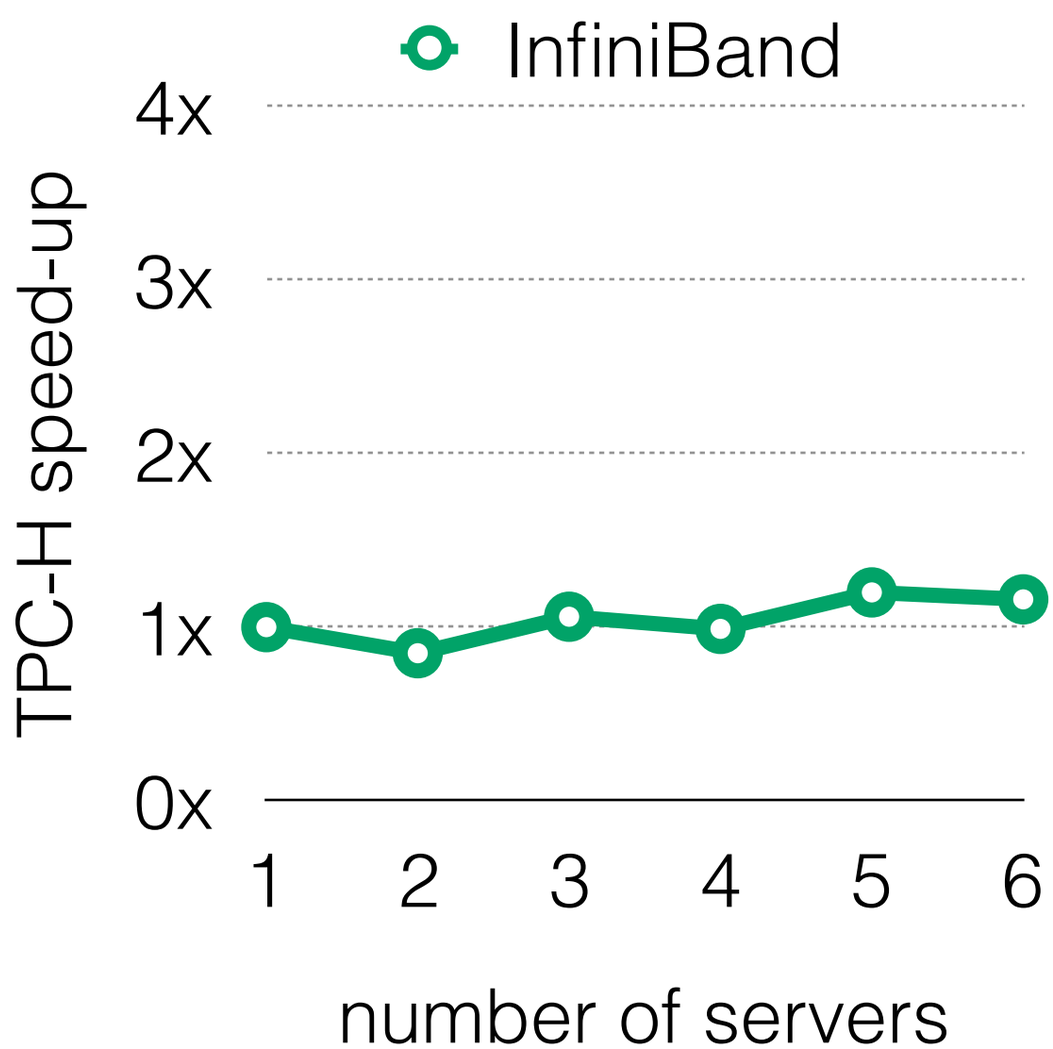
Can’t we just use faster network hardware?

- **Low bandwidth** is main bottleneck
- InfiniBand offers up to **100× the bandwidth**
- Existing software can use IPoIB **unchanged**



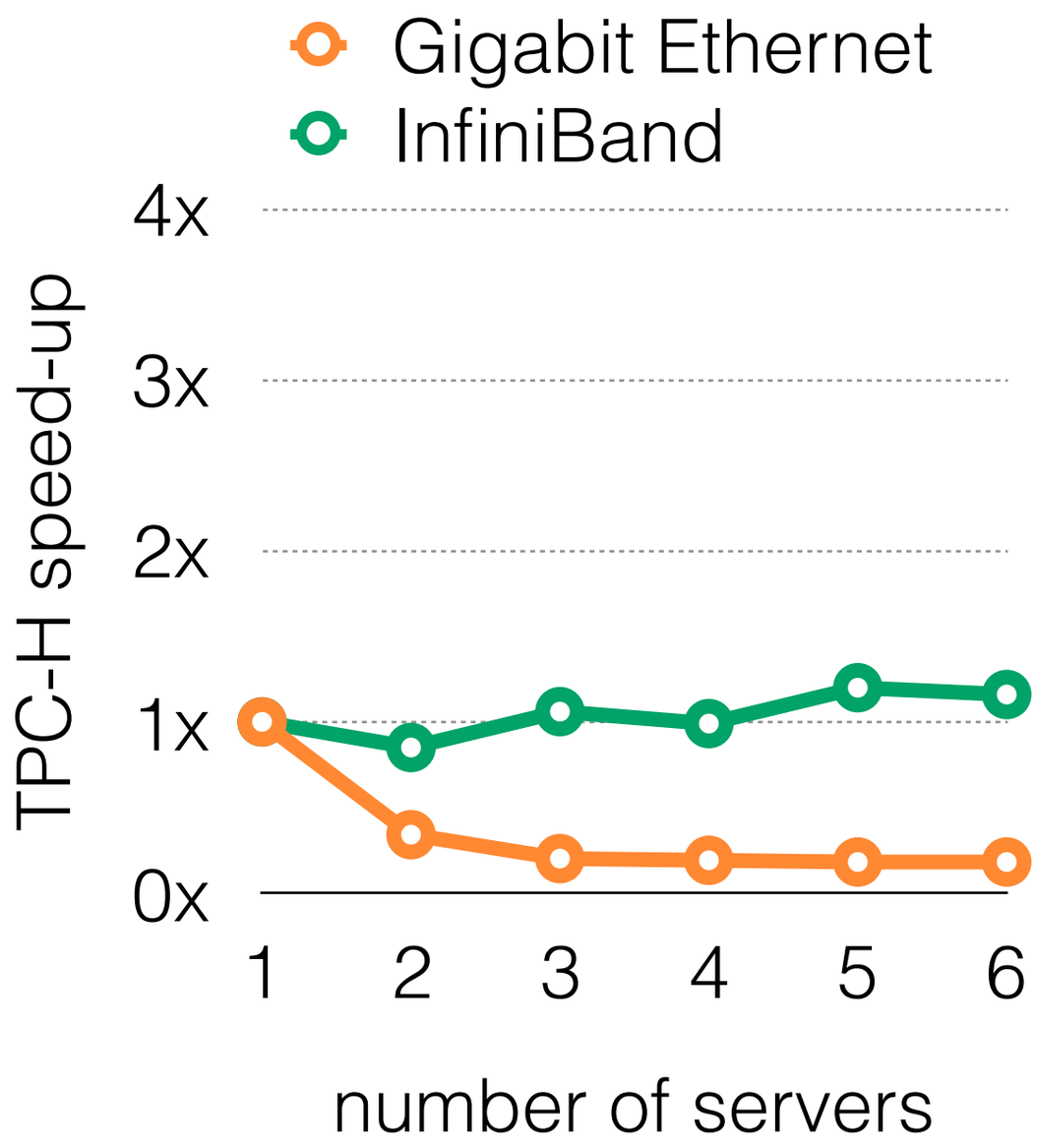
Can’t we just use faster network hardware?

- **New bottlenecks:**
  - TCP/IP stack processing
  - Interrupts
  - Context switches
  - Multiple memory transfers

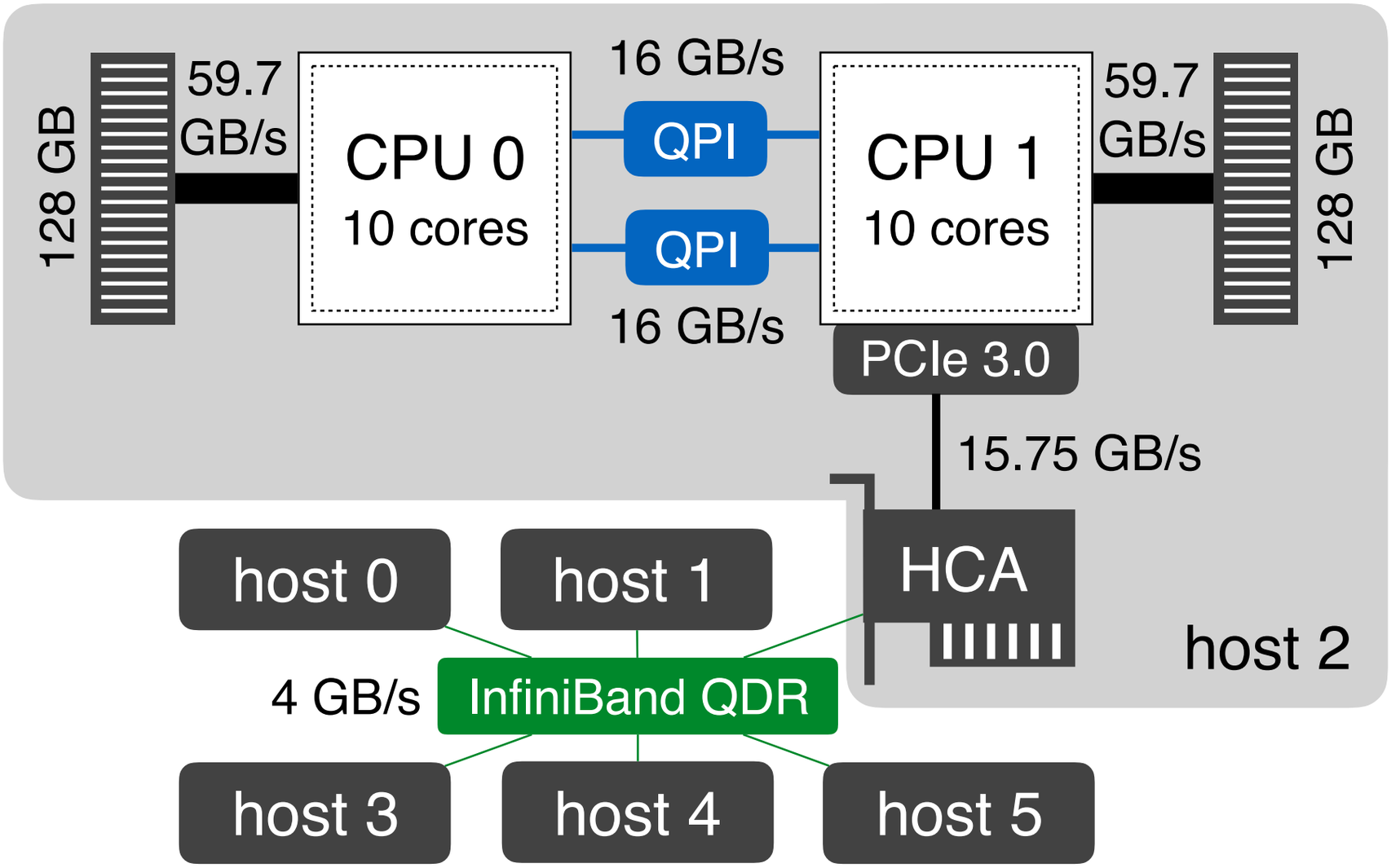


Software has to Change

- For **slow** networks more servers = less performance
- New bottlenecks emerge for **faster** networks
- **Software** has to change as well



Two Types of Networks



Two Types of Networks

QPI

- Connects **NUMA** sockets in a server
- **32 GB/s** bandwidth
- **0.2 *μs*** latency
- **Cache-coherent**

InfiniBand QDR

- Connects **servers** in a cluster
- **4 GB/s** bandwidth
- **1.3 *μs*** latency
- **Not** cache-coherent

Hybrid Parallelism

On each server:*

- Use flexible worker threads instead of **exchange** operators
- **Work stealing** per CPU
- Work stealing across **NUMA** sockets

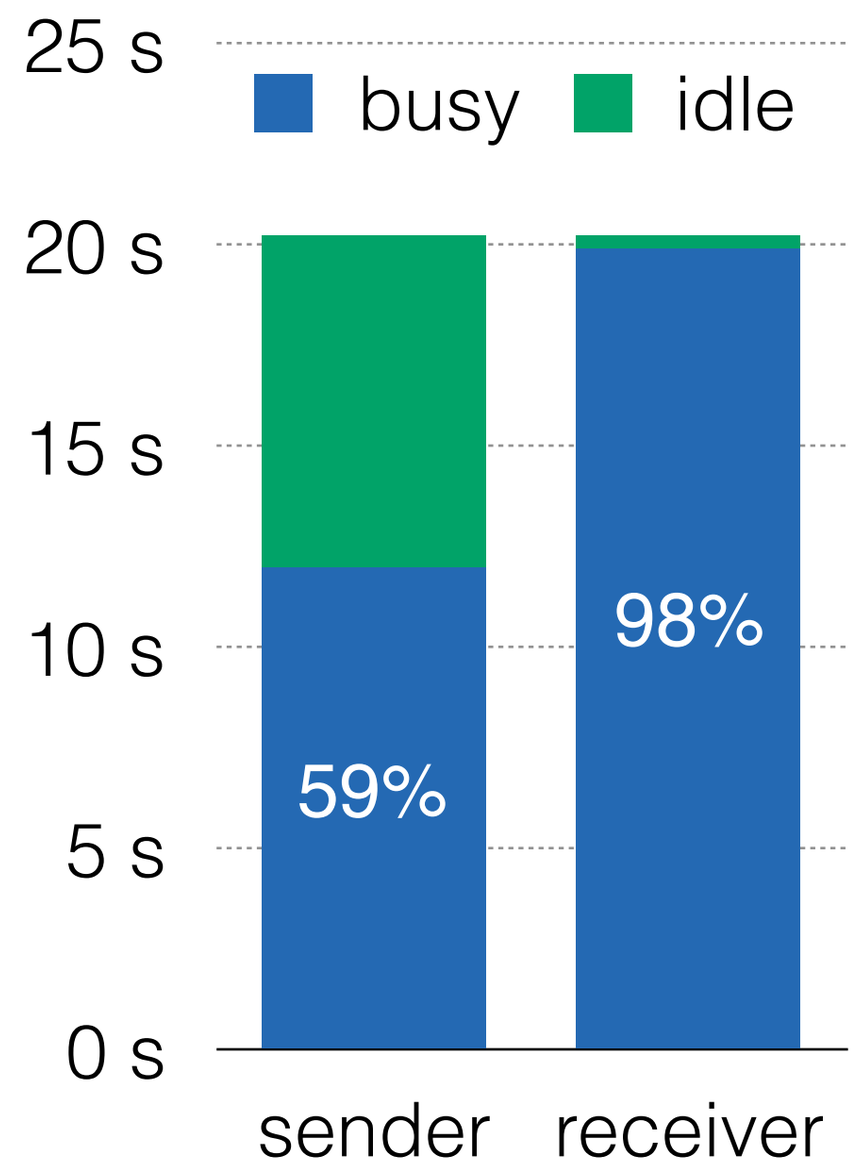
Between servers:

- Use Remote Direct Memory Access **(RDMA)** instead of TCP
- **Decoupled** exchange operators
- Network **scheduling**

* Leis et al., Morsel-driven parallelism, SIGMOD 2014

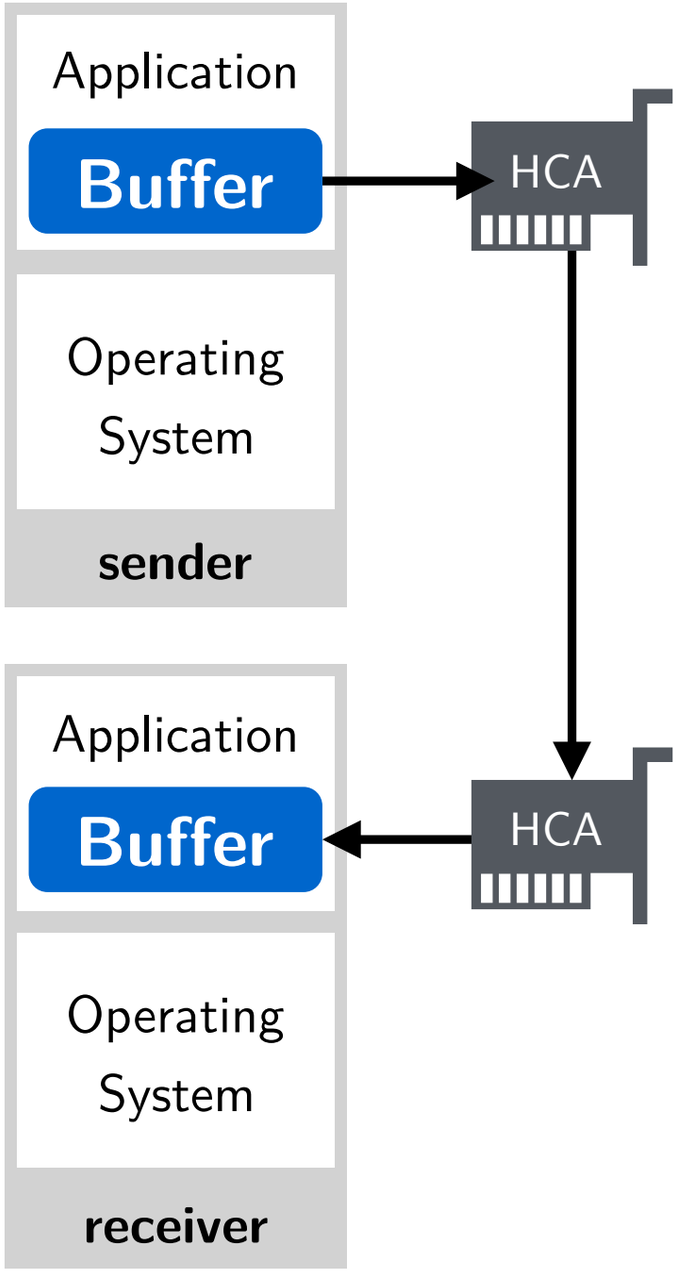
TCP over InfiniBand

- TCP is **compute-bound** at the receiver
- Even with **large MTUs** and **TCP offloading**
- Using a **separate core** for interrupts improves throughput by **53%**
- Still **compute-bound**

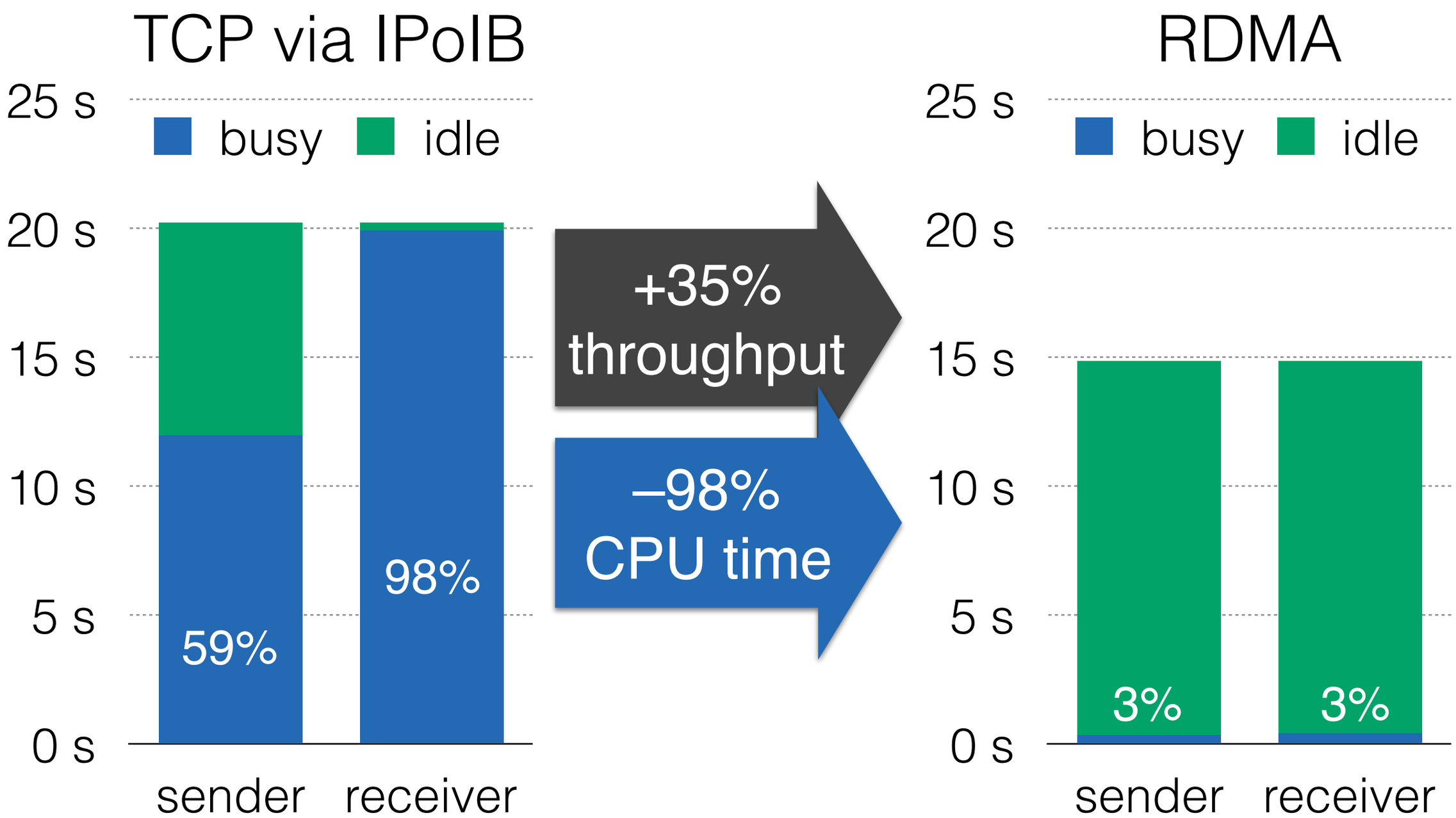


Remote Direct Memory Access

- **Bypasses** operating system and application
- **Zero-copy** network communication:
  - Achieves full **network** throughput
  - Almost no **CPU** cost
  - Less **data copying**

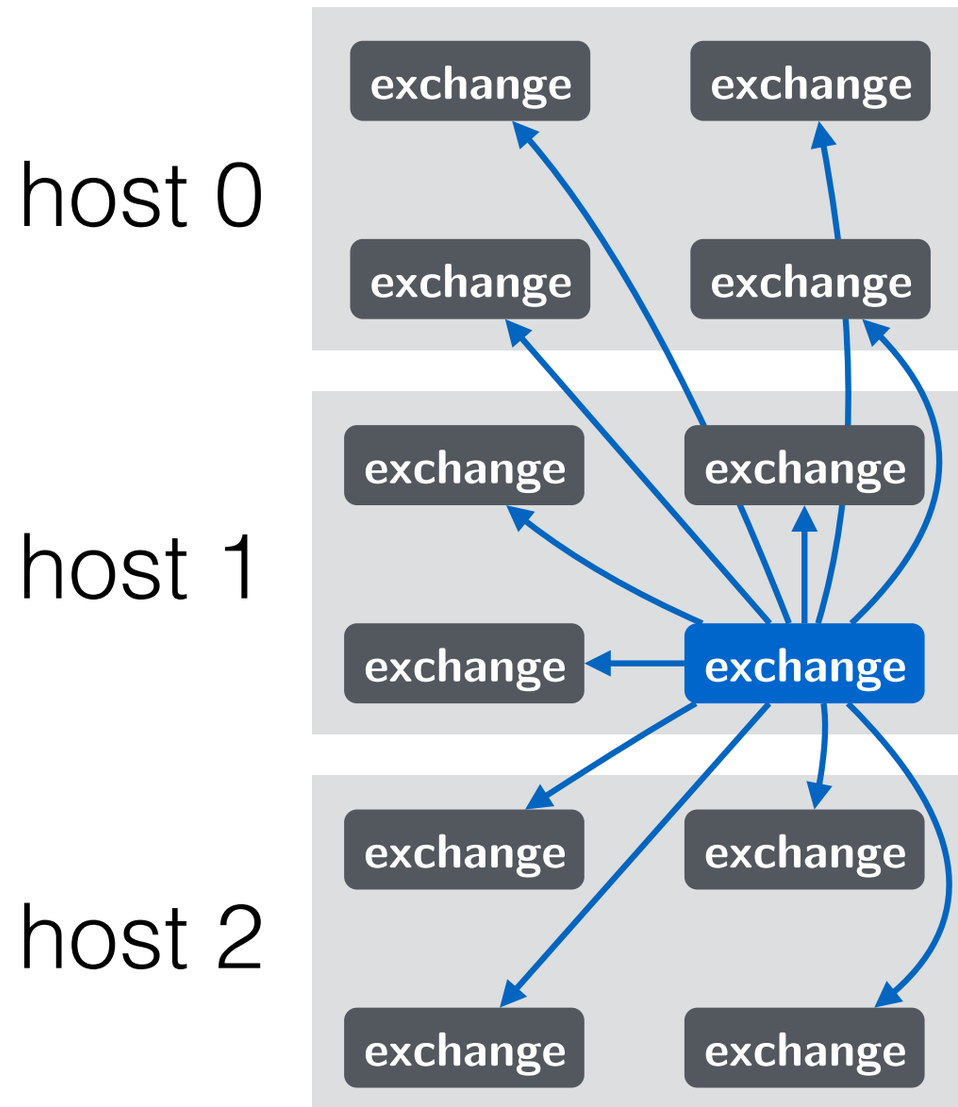


Remote Direct Memory Access



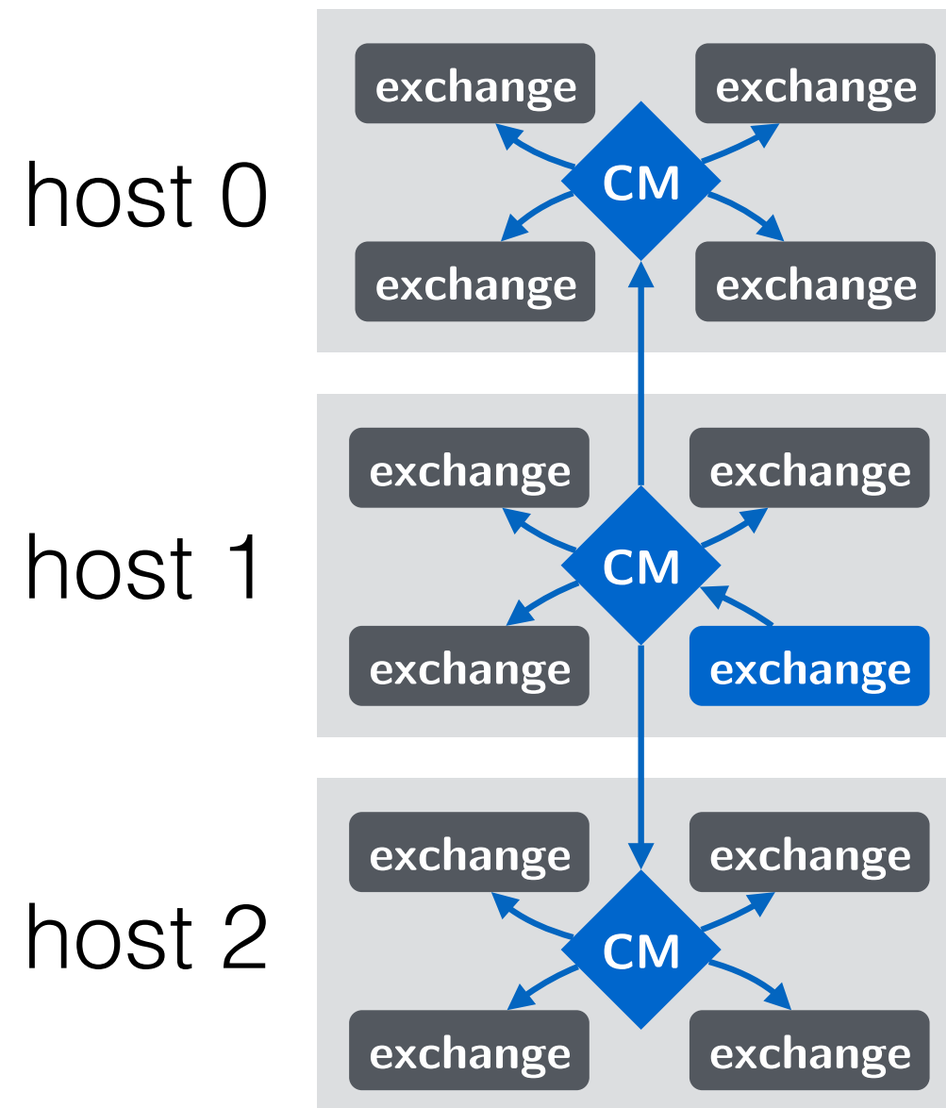
Classic Exchange

- A buffer **per exchange:**
  - # buffers per server = **servers** × **cores**^{2}
  - **1 GB**/server for 6 hosts and 20 cores
- **Skew** is huge problem:
  - Join key assigned to **fixed** exchange
  - **No work stealing**

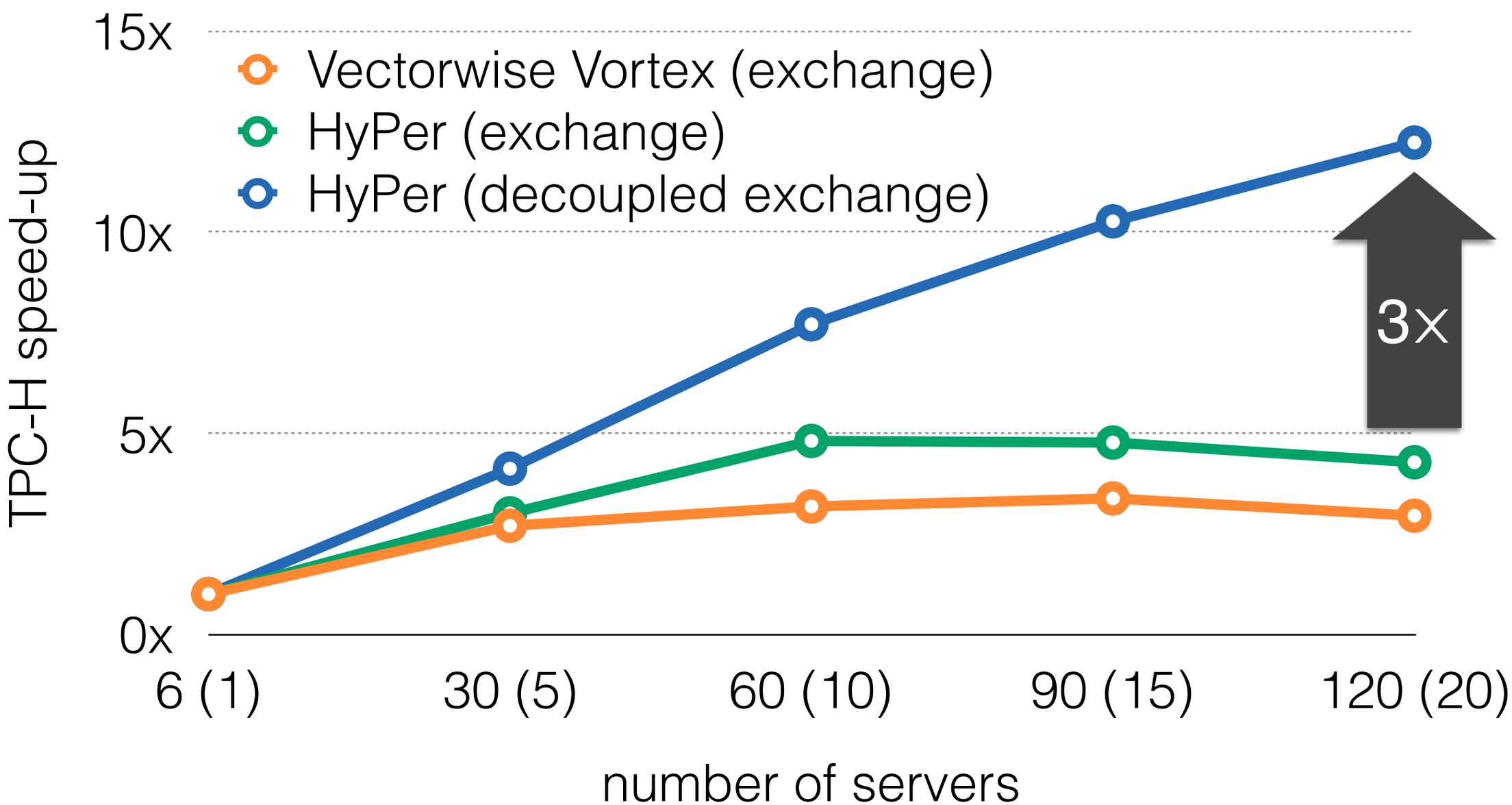


Decoupled Exchange

- Use **communication multiplexers** (CM)
- **Address servers** not individual cores:
  - Decreases **memory** consumption (2.5 MB instead of 1 GB)
  - Reduces negative impact of **skew**
  - Makes **broadcast** more applicable

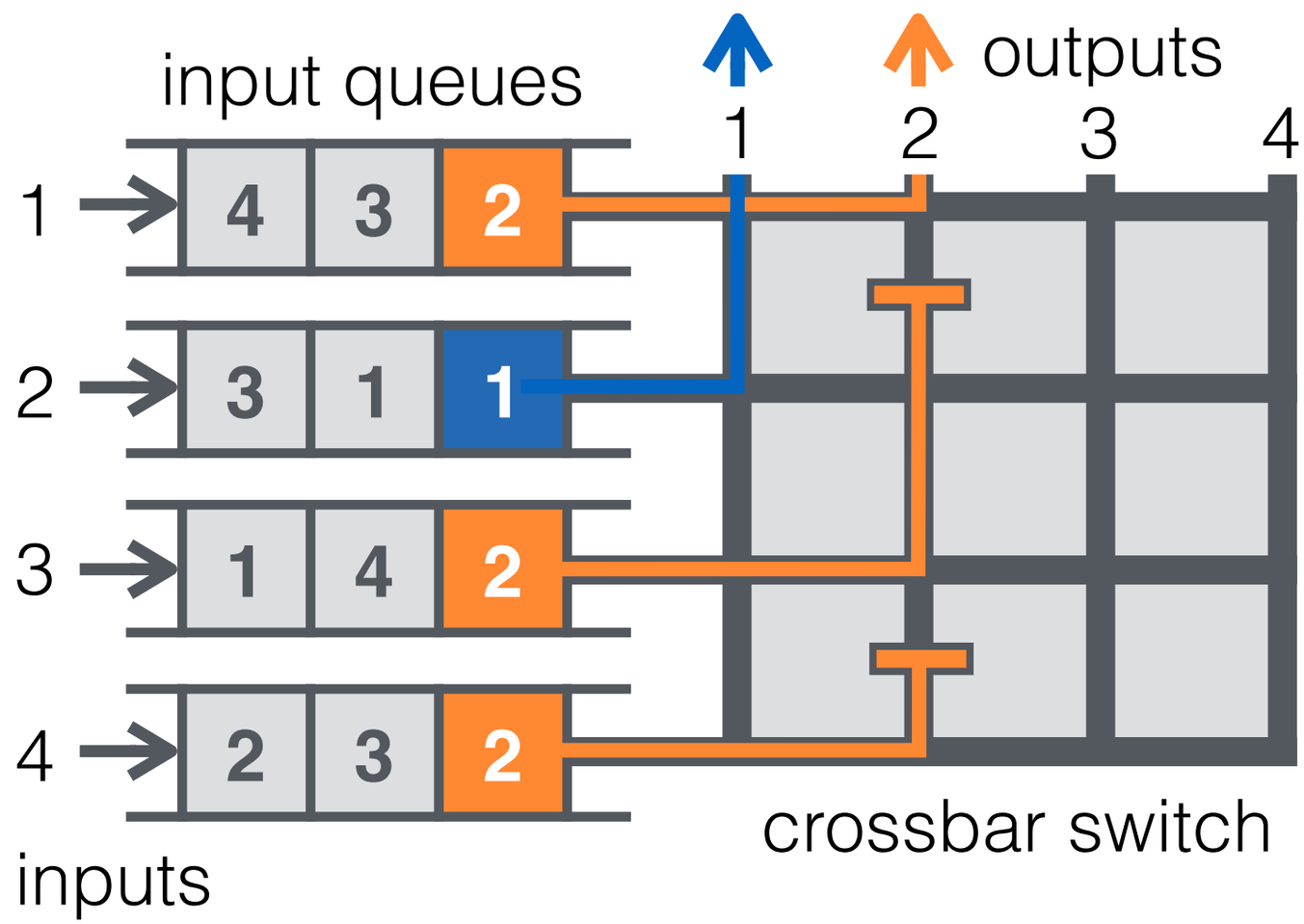


Decoupled Exchange

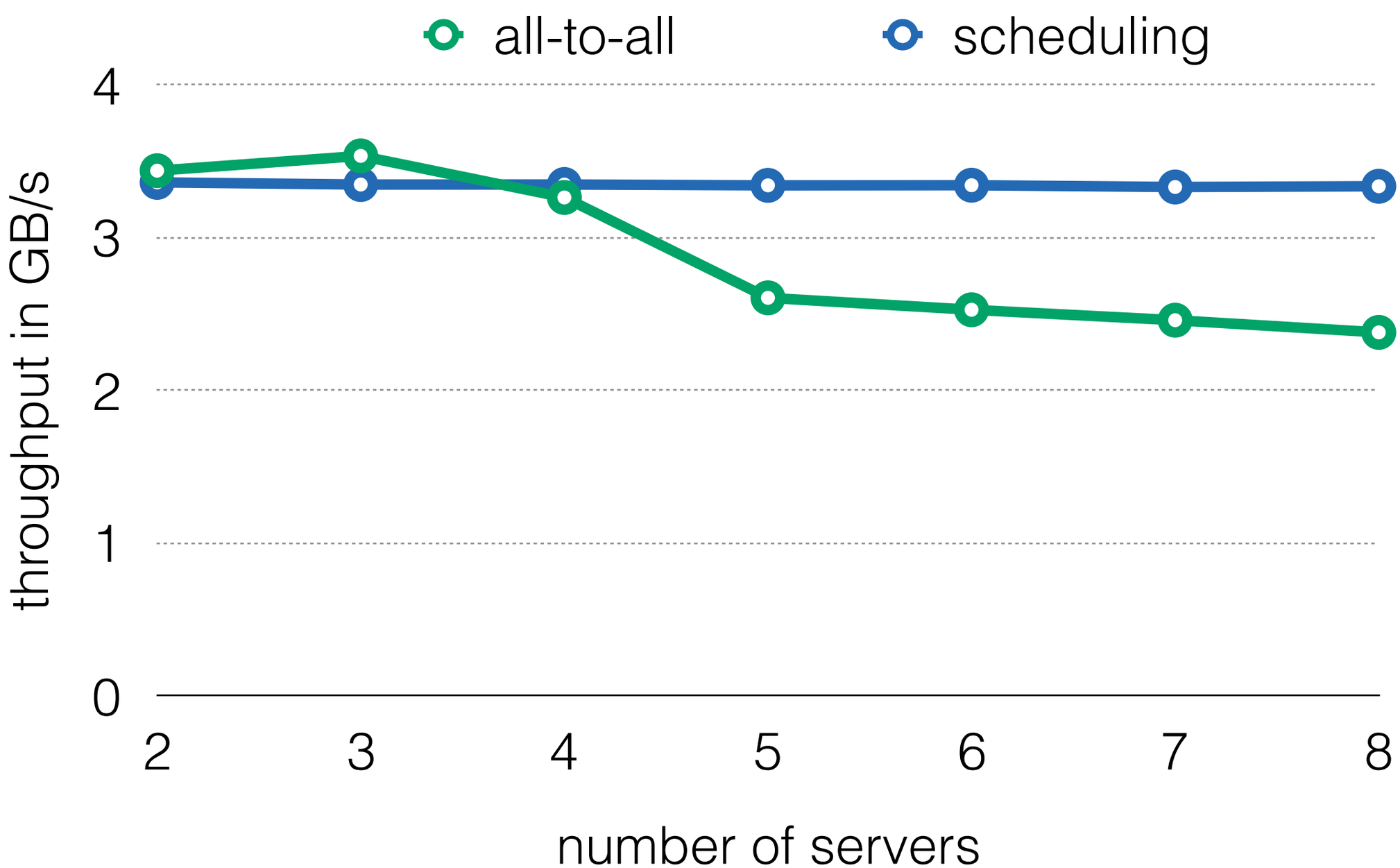


Network Scheduling

- Uncoordinated all-to-all transfers cause **switch contention**
- Make sure a server **sends** to at most one server
- **Low-latency** inline RDMA messages for network scheduling

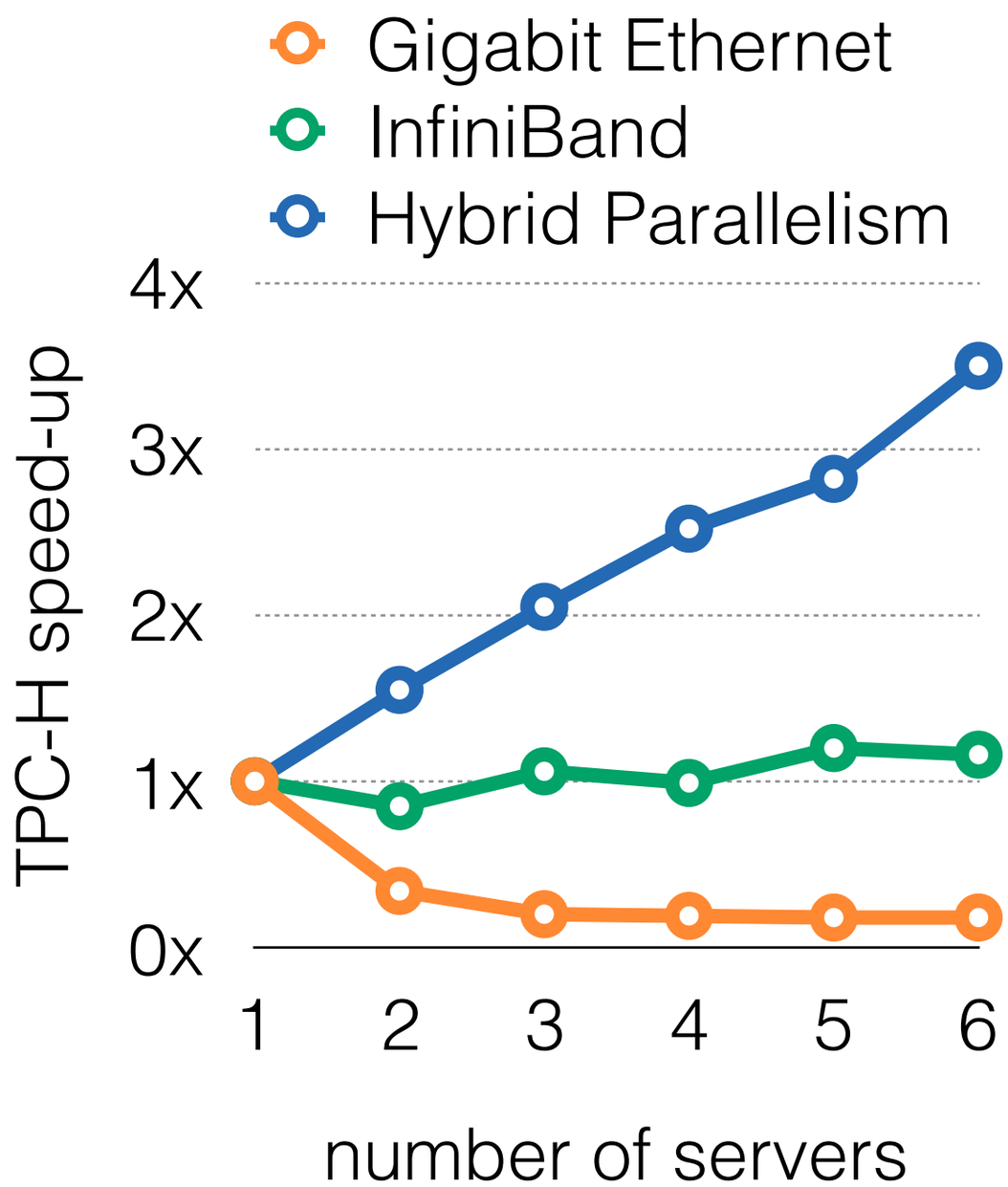


Network Scheduling

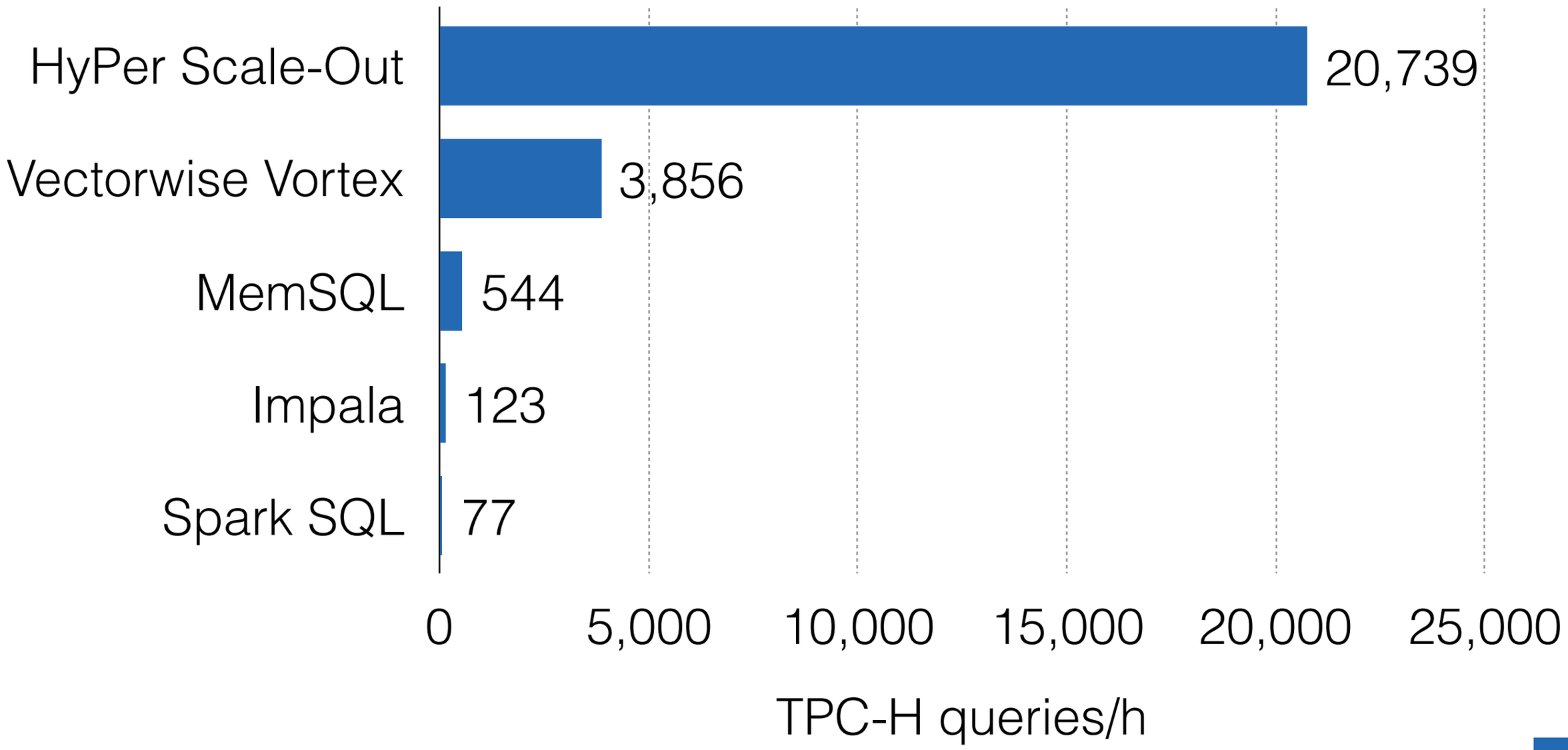


Summary

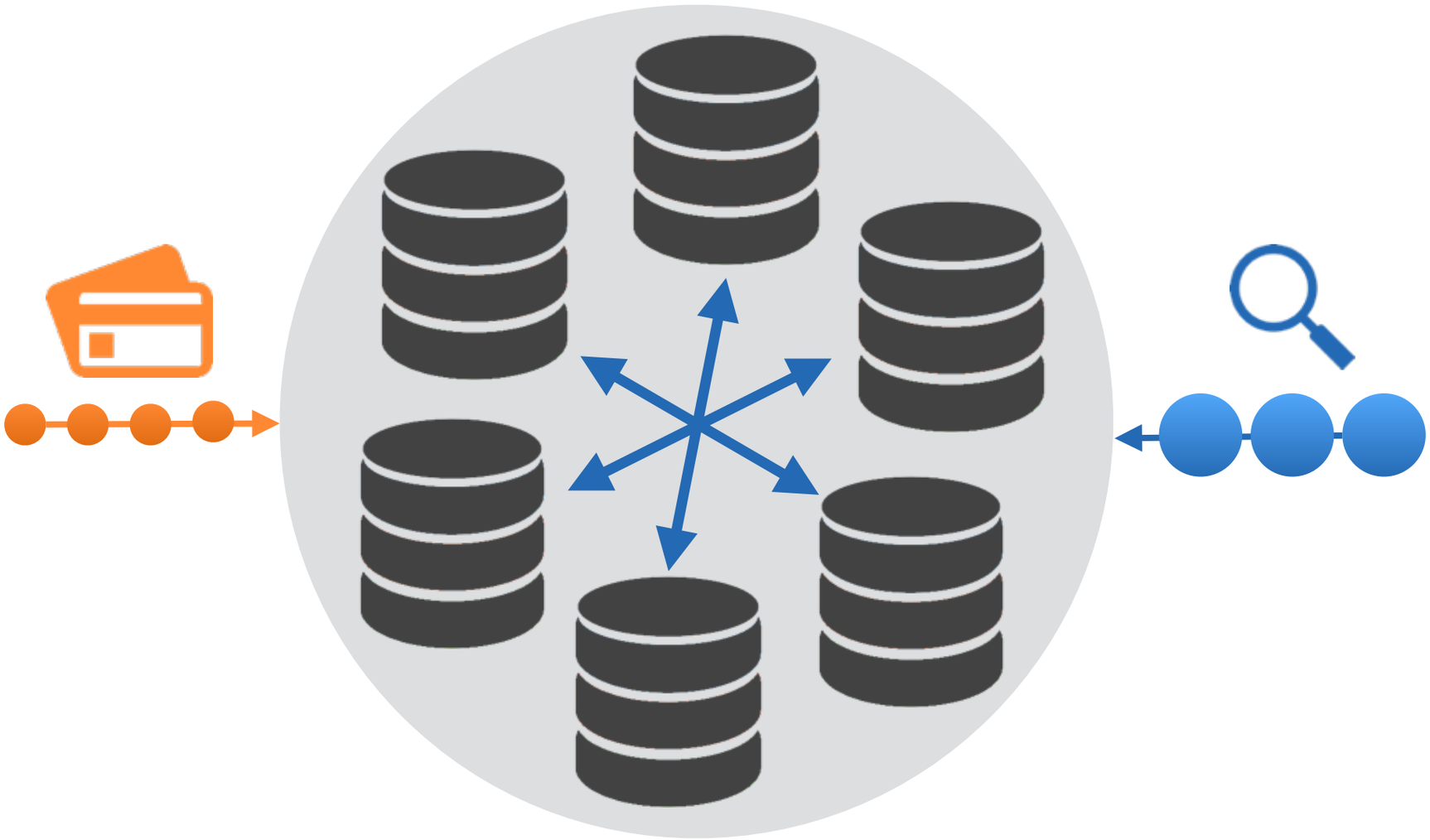
- For **slow** networks more servers = less performance
- New bottlenecks emerge for **faster** networks
- **Hybrid parallelism** optimizes for both types of networks



How do we compare?



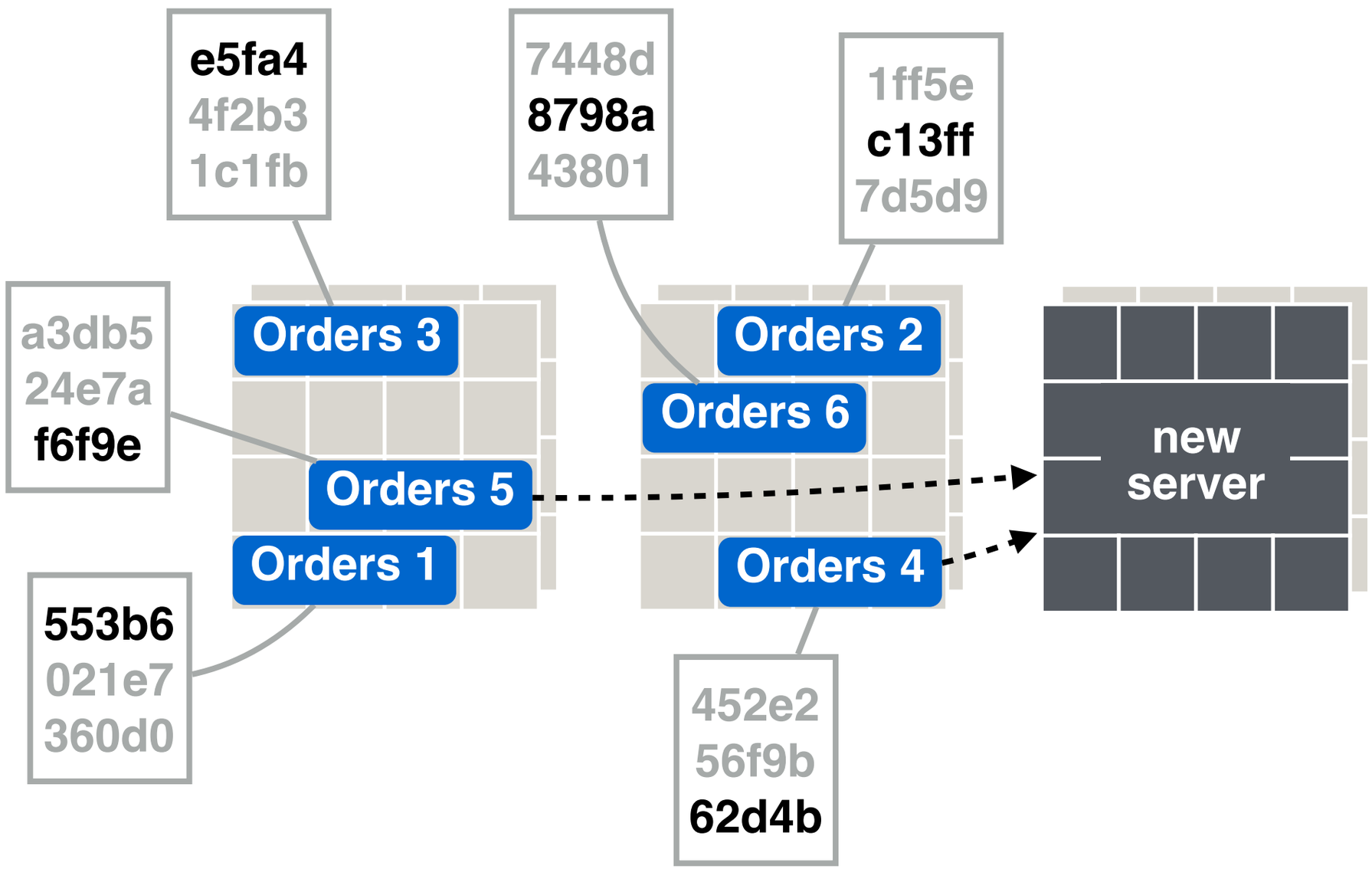
Future Work



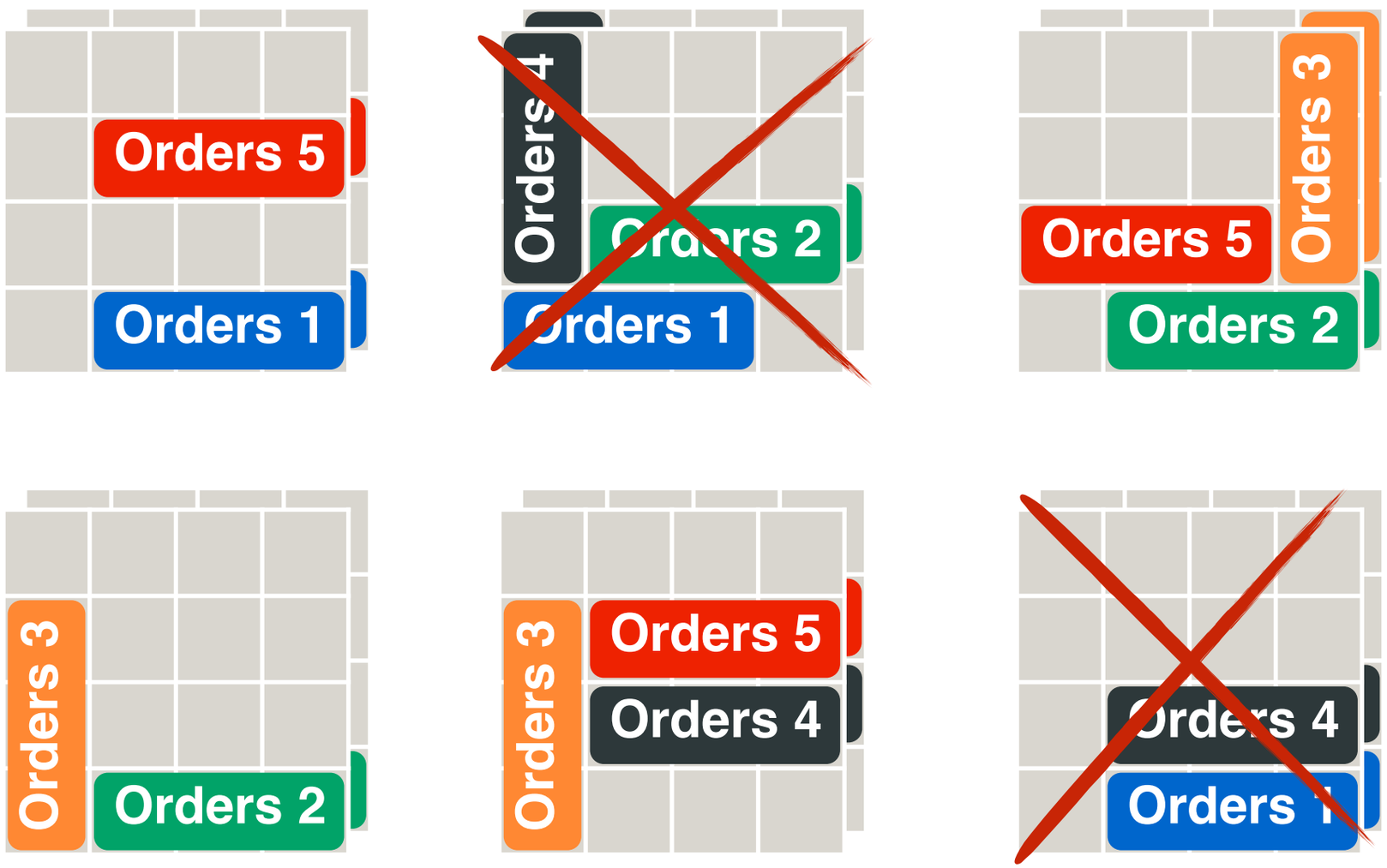
What about low-latency distributed **transactions**?

Backup

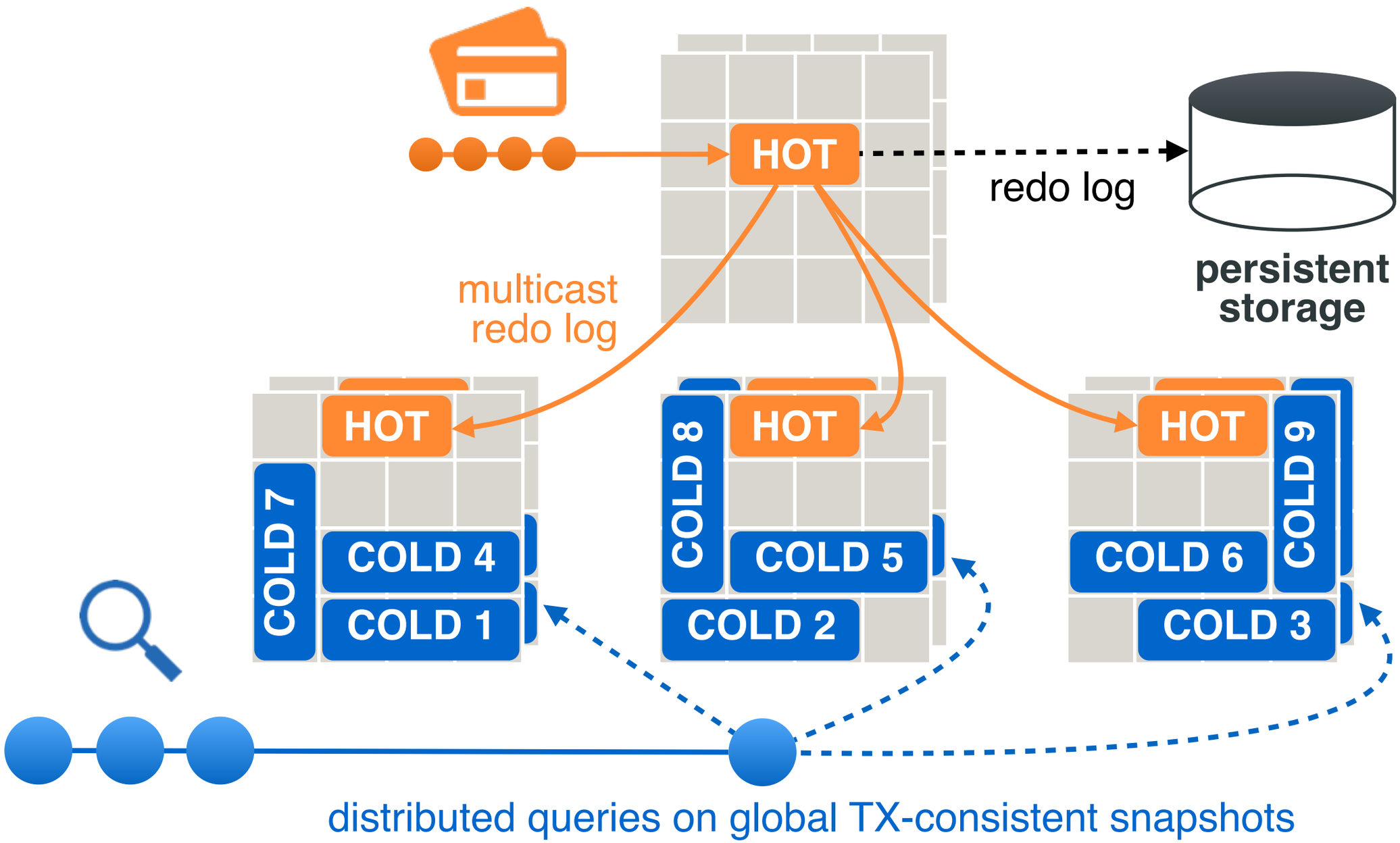
Elasticity


e5fa4
4f2b3
1c1fb
7448d
8798a
43801
1ff5e
c13ff
7d5d9
a3db5
24e7a
f6f9e
Orders 3
Orders 2
Orders 6
Orders 5
Orders 1
Orders 4
new
server
553b6
021e7
360d0
452e2
56f9b
62d4b

High Availability



Hot/cold approach



Parallelism

theory

practice